

Trend Pullback Quantitative Strategy Engine

Product Requirements, Technical Design & Research Specification — Version 1.0

September 2026

Contents

1 Document Control	5
1.1 Version History	5
1.2 How to Read This Document	5
2 Executive Summary	5
2.1 What the system is	5
2.2 The problem it solves	6
2.3 Intended user	6
2.4 What the strategy is trying to capture	6
2.5 Why pullback entries, not breakouts	6
2.6 Why trend confirmation and momentum recovery are kept separate	6
2.7 Why ATR-based, volatility-adjusted risk	6
2.8 Why long-only for Indian cash equities	7
2.9 What this system does NOT attempt to do	7
2.10 Hypothesis vs. implementation vs. backtest vs. validation vs. live trading	7
3 Strategy Hypothesis	7
3.1 Formal statement	7
3.2 Definitions used in the hypothesis (operational, not colloquial)	8
3.3 What would falsify H1	8
3.4 What would NOT falsify H1	8
4 Exact Strategy Rules	9
4.1 Trend Filter	9
4.2 RSI Entry Trigger	9
4.3 Complete Long Entry Condition	9
4.4 Entry Execution	10
4.5 Stop Loss	10
4.6 Take Profit	10
4.7 Exit Rules	10
4.8 Position Rules	11
5 Data Requirements	11
5.1 Required OHLCV fields	11
5.2 Frequency	11
5.3 Minimum historical data	12
5.4 Missing-data handling	12
5.5 Duplicate candles	12

5.6	Invalid OHLC handling	12
5.7	Corporate actions, splits, dividends, adjusted vs. unadjusted prices	12
5.8	NSE/BSE-specific considerations	13
6	Indicator Specification	13
6.1	EMA(50)	13
6.2	RSI(14) — Wilder’s method	14
6.3	ATR(14) — Wilder’s method (the default; this is not SMA-ATR or EMA-ATR)	14
7	Warm-Up Period	15
8	Backtest Engine Architecture	16
8.1	Component list and responsibilities	16
8.2	Key edge cases per component	17
9	Backtest Event Flow	18
9.1	Per-bar sequence	18
9.2	Look-ahead-bias analysis of this ordering	18
9.3	Recommended final ordering	18
10	Intrabar Ambiguity	19
10.1	The problem	19
10.2	Candidate solutions	19
10.3	Default policy for V1.0 (stated explicitly as an assumption, not hidden)	19
11	Transaction Cost Model	20
11.1	Requirement	20
11.2	Explicit non-requirement	20
11.3	Illustrative example configuration	20
11.4	Why ignoring costs distorts results, especially for short-horizon strategies	20
12	Slippage Model	21
12.1	Supported types	21
12.2	Where slippage applies	21
13	Position Sizing	21
13.1	Fixed quantity	21
13.2	Fixed rupee risk (recommended default)	22
13.3	Zero-size edge case	22
14	Capital Model	22
15	Trade Ledger	23
16	Performance Metrics	23
17	Equity Curve and Visualizations	25
18	Benchmarking	25
18.1	Required comparisons	25
18.2	Required benchmark metrics (computed identically to the strategy’s own)	26
18.3	Why benchmark selection matters, and what not to claim	26
19	Research Methodology	26
19.1	Three-way data split	26
19.2	Why repeated testing on the same data causes overfitting	26

20 Walk-Forward Testing	27
20.1 Framework	27
20.2 Parameters to define	27
20.3 What a good walk-forward result looks like	27
21 Parameter Sensitivity	28
21.1 Base parameters	28
21.2 Sensitivity sweep ranges	28
21.3 What the sweep must NOT be used for	28
21.4 What the sweep IS for: robustness and parameter-stability assessment	28
22 Overfitting Controls	28
23 Monte Carlo Analysis	29
23.1 What to randomize	29
23.2 Outputs	29
23.3 Explicit limitation	30
24 Python Implementation	30
24.1 Recommended project structure	30
24.2 Configuration schema (Python side, mirrors Section 38 YAML)	31
24.3 Example execution flow	32
25 Python Code Requirements — Representative Implementations	32
25.1 EMA (Section 5.1)	32
25.2 RSI (Section 5.2), Wilder smoothing	33
25.3 ATR (Section 5.3), Wilder smoothing	33
25.4 Signal generation (Section 3.3)	34
25.5 Position sizing (Section 12.2)	34
25.6 Trade execution / stop-target handling (Sections 3.4–3.7, 9)	34
25.7 Cost calculation (Section 10)	35
25.8 Performance metrics (Section 15, representative subset)	35
26 Java Implementation	36
26.1 Recommended project structure	36
26.2 Class relationships	37
27 Java Code Requirements — Representative Implementations	37
27.1 Candle	37
27.2 EMA (stateful, incremental — mirrors Section 5.1)	37
27.3 RSI (Wilder, mirrors Section 5.2)	38
27.4 ATR (Wilder, mirrors Section 5.3)	39
27.5 Strategy — entry signal (mirrors Section 3.3, Python 24.4)	40
27.6 Correspondence to the Python implementation	41
28 Python vs. Java — Comparison	41
29 Testing Strategy	43
29.1 Indicators	43
29.2 Strategy	43
29.3 Risk	43
29.4 Execution	44
29.5 Portfolio	44
29.6 Regression	44
30 Data Leakage and Look-Ahead Bias	44

31 Survivorship Bias	45
32 Liquidity Filter (Optional)	45
33 Market Regime Analysis	46
34 Multi-Stock Backtesting	46
34.1 Per-symbol independence	46
34.2 The critical bug this section exists to prevent	46
35 Multi-Timeframe Extension (Future Work — Not in V1.0)	47
36 Live Trading Architecture (Future Work — Not in V1.0)	47
37 Paper Trading (Future Work — Not in V1.0)	48
38 Risk Management	48
39 Configuration	49
40 Logging	50
41 Output Report	51
41.1 Required structure	51
42 Acceptance Criteria	51
42.1 Correctness	51
42.2 Performance reporting	51
42.3 Reproducibility	51
42.4 Testing	52
42.5 Auditability	52
42.6 What is explicitly NOT an acceptance criterion	52
43 Project Milestones	52
44 Versioning	53
45 Research Questions	54
46 Failure Conditions	54
47 Security and Reliability	55
48 Reproducibility	56
49 Final Project Directory (Both Languages)	56
50 Example End-to-End Run (Hypothetical Numbers Only)	57
51 Final Architecture Diagram	58
52 Glossary	58
A Full Illustrative Configuration Reference	59
B Items Marked TBD in This Specification	60
References	61

1 Document Control

Field	Value
Document type	Combined PRD / Technical Design / Quant Strategy Spec / Research Protocol
Strategy name	Trend Pullback Quantitative Strategy Engine
Version	1.0
Status	Draft for implementation
Scope	Long-only, single-symbol daily-bar equity backtesting (India cash market context)
Author	Quant Engineering (specification author unspecified — fill in before distribution)
Reviewers	TBD

1.1 Version History

Version	Change
0.1	Initial draft — strategy rules and indicator definitions only
1.0	Full specification: architecture, cost model, validation protocol, Python/Java specs

1.2 How to Read This Document

This is a specification, not a pitch. It defines what must be built and how it must behave. It does not claim the strategy is profitable — that is an open empirical question this document sets up a protocol to answer. Anywhere a number is not derivable from first principles (broker fee schedules, specific data vendor behavior, live infrastructure choices), it is marked **TBD** or flagged as **illustrative/configurable**, not asserted as fact. Do not treat any illustrative number in this document as a production value without checking it against the current, authoritative source.

Three words recur throughout and are used precisely, not loosely: *hypothesis* (untested claim), *backtest* (result on historical data using a specific set of assumptions), *validation* (the process of checking whether a backtest result is likely to generalize). Nothing in Sections 1–22 should be read as a live trading recommendation.

2 Executive Summary

2.1 What the system is

A backtesting and research engine for one specific, narrowly defined trading rule: buy an equity when it is in an established uptrend, has just pulled back on momentum, and is showing the first sign of a bounce, measured by RSI(14) crossing back above 40. Risk is capped with an ATR-based stop, and the target is a fixed 2R multiple. The system’s job is to simulate this rule honestly on historical OHLCV data — including costs, slippage, and out-of-sample checks — and produce a report that a competent reviewer can audit line by line.

2.2 The problem it solves

Most retail and semi-professional “strategy backtests” fail for a small number of repeat reasons: look-ahead bias (using information not available at decision time), ignoring transaction costs, testing and re-testing on the same data until something looks good (overfitting), and reporting metrics without confidence intervals or out-of-sample checks. This specification exists to prevent all four failure modes by construction — not by reminding the developer to be careful, but by defining an architecture and event order where the mistakes are structurally difficult to make.

2.3 Intended user

A single developer or small team building a research-grade backtester for personal or small-fund use, comfortable with Python and/or Java, who wants a rule-based system rather than a discretionary one, and who wants the codebase to eventually support (but not yet include) live paper trading.

2.4 What the strategy is trying to capture

The hypothesis is a momentum-continuation effect inside an established trend: price does not move in a straight line, and short-term pullbacks inside an uptrend are sometimes followed by a resumption of the trend rather than a reversal. The RSI reclaim of 40 is used as an operational (not theoretical) definition of “the pullback is over.” This is a statistical bet, not a law of markets.

2.5 Why pullback entries, not breakouts

Breakout systems buy strength (price exceeding a prior high) and therefore buy after the market has already moved, at a point where slippage and false-breakout risk are highest. A pullback system buys weakness inside a still-intact uptrend, which — if the hypothesis holds — offers a better average entry price and a naturally short distance to a logical stop (the recent low), improving the risk/reward per trade. The cost is fewer signals, and the same trend-continuation assumption used by breakout systems (both fail the same way in choppy, non-trending markets).

2.6 Why trend confirmation and momentum recovery are kept separate

The strategy uses two independent filters — a slope-and-position trend filter on EMA(50), and an RSI(14) reclaim trigger — rather than a single blended indicator. This is deliberate. A trend filter answers “is this a market where buying dips is a reasonable idea at all?” A momentum trigger answers “has the dip plausibly ended, right now?” Collapsing these into one indicator (e.g., a single stochastic-RSI-MACD composite) makes the strategy harder to reason about, harder to debug, and harder to attribute performance to a specific cause. This document explicitly rejects “indicator soup”: every indicator used has one job, and no indicator is included because it might help.

2.7 Why ATR-based, volatility-adjusted risk

A fixed rupee or fixed percentage stop is blind to how volatile the instrument currently is: the same stop distance is too tight for a volatile stock (stopped out by noise) and too loose for a quiet one (undersized risk control). ATR(14) measures the stock’s own recent volatility in its own units, so the stop distance adapts per trade and per stock without manual tuning.

2.8 Why long-only for Indian cash equities

The base version assumes trading Indian cash-market equities (NSE/BSE) where short selling intraday-only rules, delivery margin requirements, and securities lending frictions make short selling materially more operationally complex than going long. Restricting V1.0 to long-only removes an entire class of implementation and cost-modeling complexity so the core signal can be validated first. This is a scoping decision, not a claim that shorting is unprofitable.

2.9 What this system does NOT attempt to do

- It does not predict prices. It defines a rule and measures what the rule would have done.
- It does not claim an edge exists. Section 44 (“Research Questions”) and Section 45 (“Failure Conditions”) are exactly as important as the strategy rules themselves.
- It does not trade live in V1.0. There is no order routing, no broker integration, no real-money risk in this version.
- It does not handle options, futures, currencies, or short selling.
- It does not use machine learning, regime-switching models, or any parameter that is fit rather than chosen a priori — walk-forward re-fitting (Section 19) is the one exception, and it is bounded and logged.
- It does not guarantee the strategy survives transaction costs, slippage, or out-of-sample data. That is what the validation protocol in Sections 18–22 exists to find out.

2.10 Hypothesis vs. implementation vs. backtest vs. validation vs. live trading

These are five different things and this document keeps them in five different sections on purpose:

Stage	What it means	Where it’s covered
Hypothesis	An untested, falsifiable claim about market behavior	Section 2
Implementation	Code that correctly executes the rule as specified	Sections 3, 5, 23–26
Backtest	A single historical simulation under one set of assumptions	Sections 7–17
Validation	Checking whether the backtest result is likely to hold up (OOS, walk-forward, sensitivity, Monte Carlo)	Sections 18–22
Live trading	Real capital, real fills, real latency	Explicitly out of scope for V1.0 — Section 35

A correct implementation of a bad hypothesis will still produce a losing backtest. A profitable backtest of a correctly implemented strategy can still fail out-of-sample. Confusing any two of these five stages is the single most common way quant projects mislead their own authors.

3 Strategy Hypothesis

3.1 Formal statement

H1: In an equity that is in an established upward trend (defined operationally in Section 3.1), a short-term weakening of upward momentum followed by a recovery

of that momentum (defined operationally as RSI(14) crossing back above 40 from below) marks a statistically favorable long entry point, when risk is sized as a multiple of recent volatility (ATR(14)) and the position is closed at either a fixed stop or a fixed 2R target.

3.2 Definitions used in the hypothesis (operational, not colloquial)

- **Market regime / trend:** Not “the stock looks like it’s going up.” Specifically: EMA(50) has risen over the last 5 bars AND price is currently above EMA(50). Nothing else counts.
- **Pullback:** Not a price decline of any specific magnitude. Operationally represented indirectly, through RSI(14) having dropped below 40 at some point before the current bar.
- **Momentum recovery:** RSI(14) crossing from below 40 to at-or-above 40 on the current bar (Section 3.2). This is a proxy for “selling pressure within the pullback has eased,” not a claim that price has bottomed.
- **Volatility:** ATR(14), Wilder-smoothed (Section 5).
- **Risk/reward:** Defined structurally, not discovered empirically — $1.5 \times \text{ATR}$ stop, 2R target ($3 \times \text{ATR}$), fixed before entry, not adjusted afterward in V1.0.

3.3 What would falsify H1

The hypothesis is falsified — not “the code has a bug,” but the *idea itself* is wrong or too weak to trade — if, after honest implementation and full validation (Sections 18–22), any of the following hold:

1. Expectancy per trade is at or below zero after realistic transaction costs and slippage.
2. Profit factor is materially below 1.0 in out-of-sample data even if it looks acceptable in-sample.
3. Performance does not survive a walk-forward test — i.e., parameters fit on one window do not generalize to the next.
4. Results are extremely sensitive to small parameter changes (e.g., RSI level 39 vs 41 flips the strategy from profitable to unprofitable) — this indicates curve-fitting to noise, not a real effect.
5. Almost all net profit comes from a small number of outlier trades (Section 44, Q5) rather than a broad-based edge.
6. Monte Carlo resampling of the trade sequence shows an unacceptable probability of ruin or drawdown at the position sizing used.

None of these are hypothetical embarrassments to be argued away. Any one of them occurring is a valid, useful research result: it means H1 does not hold in the tested form. Section 45 expands this into explicit failure conditions with matching required actions.

3.4 What would NOT falsify H1

- A single losing trade, or a losing streak within a range consistent with the strategy’s own historical win rate and Monte Carlo bounds.
- Underperformance in a single, clearly-labeled adverse regime (e.g., a broad bear market), *provided* the strategy’s trend filter correctly kept it flat or mostly flat during that period. A long-only trend-following pullback system is not expected to make money when there is no uptrend to pull back within.
- Sensitivity of the exact win rate or profit factor to minor cost assumptions, as long as the sign and order of magnitude of the edge is stable.

4 Exact Strategy Rules

Every condition below is written so that two independent developers, given the same OHLCV series, produce bit-identical (or floating-point-tolerance-identical) signal series. Any deviation from these definitions is a bug, not a design choice, unless explicitly flagged as configurable.

4.1 Trend Filter

Let $\text{EMA50}[t]$ be the 50-period exponential moving average of the close price, at bar index t .

Trend condition (both required):

$$\begin{aligned}\text{EMA50}[t] &> \text{EMA50}[t - 5] \\ \text{Close}[t] &> \text{EMA50}[t]\end{aligned}$$

Both must be true on the same bar t . This is **not** an EMA crossover strategy (no comparison of EMA50 against a second, faster EMA), and it is **not** satisfied by comparing EMA50 to the previous bar's close. Do not implement it as $\text{EMA50}[t] > \text{Close}[t-1]$ — that is a different, unintended condition and produces a different signal series.

Rationale for the 5-bar slope check: requiring the EMA to have net-risen over 5 bars (roughly one trading week on daily bars) filters out cases where price is only barely, recently, and possibly noisily above a flat or still-declining EMA — i.e., it distinguishes “in an uptrend” from “just crossed above the average.”

4.2 RSI Entry Trigger

Let $\text{RSI14}[t]$ be Wilder's RSI(14) at bar t (formula in Section 5.2).

Reclaim condition:

$$\text{RSI14}[t - 1] < 40 \quad \text{AND} \quad \text{RSI14}[t] \geq 40$$

Boundary handling — this must be specified, not left ambiguous: the condition uses strict $<$ on the prior bar and non-strict $\geq$ on the current bar. Concretely:

- If $\text{RSI}[t-1] == 40.0$ exactly: the prior-bar condition is **false** (strict less-than fails), so no signal, even if RSI is now above 40. The strategy requires having been *below* 40 first.
- If $\text{RSI}[t] == 40.0$ exactly: the current-bar condition is **true** (non-strict). A close reclaim to exactly 40.0 counts as a trigger.

This asymmetric boundary (strict on the exit side of the dip, inclusive on the recovery side) is the specified default. It is a modeling choice, not a mathematical necessity — an implementer with a strong reason to use $\leq$ instead must document the deviation and re-run all downstream validation, because it changes which exact bars generate signals.

4.3 Complete Long Entry Condition

A long signal fires at bar t if and only if **all four** conditions hold simultaneously:

$$\begin{aligned}\text{EMA50}[t] &> \text{EMA50}[t - 5] \\ \text{Close}[t] &> \text{EMA50}[t] \\ \text{RSI14}[t - 1] &< 40\end{aligned}$$

$$RSI14[t] \geq 40$$

There is no “OR” anywhere in this rule set. All four are AND-ed.

4.4 Entry Execution

Default assumption: signal confirmed at the close of bar t ; order filled at the open of bar $t + 1$.

This is the single most important anti-look-ahead rule in this document. Explicitly prohibited: entering a trade at the close of bar t using the close of bar t as the fill price, because that close price is only known at the exact instant the signal itself becomes known — no real order could have been placed and filled at that price before the information existed. Any backtest that fills at $\text{Close}[t]$ for a signal computed from $\text{Close}[t]$ is look-ahead-biased by construction, full stop, regardless of how good the resulting numbers look.

Filling at $\text{Open}[t+1]$ is itself an assumption, not a guarantee of realism — the open can gap significantly from the prior close, especially in Indian small/mid-cap names, and this gap risk is a real cost the backtest must not hide. Section 11 (Slippage) applies to this fill.

4.5 Stop Loss

$\text{ATR14}[t]$ — Wilder’s $\text{ATR}(14)$, formula in Section 5.3

$$R_{\text{per share}} = 1.5 \times \text{ATR14}[t]$$

$$\text{Stop} = \text{Entry} - 1.5 \times \text{ATR14}[t]$$

The ATR value used is the value calculated **as of the signal bar t** (not recalculated at entry on $t + 1$), so that the stop distance is fixed and known before the entry order is even placed. This is a deliberate design choice for determinism and auditability; it must not silently drift to “ATR as of entry bar” in the implementation.

4.6 Take Profit

Default reward:risk = 2R.

$$\text{Target} = \text{Entry} + 2R = \text{Entry} + 2 \times (1.5 \times \text{ATR14}[t]) = \text{Entry} + 3 \times \text{ATR14}[t]$$

So concretely: **Target = Entry + 3×ATR14[t]**. This is algebraically identical to “2× the risk distance,” restated in ATR units purely so the relationship between stop and target is visible at a glance: the target is always exactly twice as far from entry as the stop, in the same volatility-adjusted units.

4.7 Exit Rules

The base version (V1.0) has exactly four exit paths, and no others:

1. **Stop-loss hit** — $\text{Low}[\text{bar}] \leq \text{Stop}$ while position is open $\rightarrow$ exit at Stop (subject to the intrabar policy in Section 9).
2. **Take-profit hit** — $\text{High}[\text{bar}] \geq \text{Target}$ while position is open $\rightarrow$ exit at Target (subject to Section 9).
3. **End-of-data exit** — if the backtest data ends while a position is still open, close it at the last available close price and flag the trade as `exit_reason = END_OF_DATA` in the ledger. This is not a real trading signal; it exists so the equity curve and metrics reconcile, and such trades should be identifiable and optionally excluded from headline statistics.

4. **No signal-based exit** — there is no rule that closes a position because the trend filter turns false, RSI does something else, or a fixed holding period elapses. The only ways out are stop, target, or end-of-data.

Explicitly excluded from V1.0 (future work, Section 43): trailing stops, partial profit-taking, breakeven-stop moves, time-based exits, and any discretionary override. Adding any of these changes the strategy being tested and must be versioned as a new strategy variant (e.g., V1.3), not silently folded into V1.0’s results.

4.8 Position Rules

- **Long-only.** No short selling in V1.0.
- **One position at a time**, per symbol. No pyramiding (adding to a winning position), no averaging down (adding to a losing position).
- **No simultaneous positions** in the single-symbol backtester (Section 33 defines how this generalizes across symbols, where each symbol independently obeys this same one-position rule).
- **No leverage** unless explicitly enabled via a future configuration flag (off by default; Section 13).
- **Position size is configurable** (Section 12), not hardcoded.

Rationale: each of these constraints removes a degree of freedom that would otherwise let the backtest’s apparent performance depend on position-management choices layered on top of the entry/exit signal, which is the thing actually being tested. Pyramiding, averaging down, and leverage all change the risk profile in ways that can make a mediocre signal look good (or a good signal look catastrophic) independent of whether the entry/exit logic itself has an edge. Isolate the signal first; add position-management sophistication later, deliberately, as its own tested variant.

5 Data Requirements

5.1 Required OHLCV fields

Field	Type	Notes
Timestamp	date or datetime	Bar close time convention must be fixed and documented per data source
Open	float	> 0
High	float	≥ Open, Close, Low
Low	float	≤ Open, Close, High
Close	float	> 0
Volume	integer/float	≥ 0

5.2 Frequency

- **Required for V1.0:** daily (end-of-day) bars.
- **Supported architecture-wise but not validated in V1.0:** any fixed-interval intraday frequency (e.g., 15-minute, hourly), since the indicator math is frequency-agnostic. Using intraday data changes the practical meaning of “50-period EMA” (50 bars of 15-minute data ≈ a few trading days, not 50 calendar days) and requires re-validating the hypothesis at that frequency — it is not a free substitution.

5.3 Minimum historical data

At minimum: `EMA_period + slope_bars` bars for the trend filter to be defined (55 bars with default parameters), plus enough additional history that `RSI(14)` and `ATR(14)` are past their own warm-up (Section 6). In practice, request materially more than the minimum — at least 3–5× the longest indicator period — so that early-bar values are not distorted by initialization transients (Section 5).

5.4 Missing-data handling

- **Missing bars (gaps in the calendar):** do not forward-fill price data to manufacture a bar for a day the exchange was closed or the symbol did not trade — this fabricates information. Only forward-fill if the *data source* dropped a bar the exchange actually had (a data-integrity, not a market, gap), and log every such fill.
- **Missing fields within a bar** (e.g., volume null): reject the bar for indicator calculation purposes unless the field is not required for enabled indicators; never silently substitute zero for price fields.
- **NaN propagation:** any indicator computed from a window containing a missing/rejected bar must itself be NaN until the window is clean; the strategy must never generate a signal from a NaN indicator value.

5.5 Duplicate candles

Two rows with the same timestamp for the same symbol: reject the batch and raise a data-quality error rather than silently picking one (first, last, or an average) — silently resolving duplicates hides a data pipeline bug that could also be corrupting other rows undetected.

5.6 Invalid OHLC handling

A bar fails validation (and must be excluded, not clamped) if any of:

- `High < Low`
- `High < Open` or `High < Close`
- `Low > Open` or `Low > Close`
- Any of `Open/High/Low/Close/Volume` is negative
- `Open/High/Low/Close` is zero (except explicitly for instruments where zero is valid, which does not apply to equities)

Log every rejected bar with symbol, timestamp, and the specific rule violated. A high rejection rate for a given symbol/source is itself a signal that the data source is unreliable for that symbol.

5.7 Corporate actions, splits, dividends, adjusted vs. unadjusted prices

- **Stock splits/bonuses** change the share count and price scale discontinuously. If back-testing on unadjusted prices across a split date, `EMA/RSI/ATR` will show a spurious, enormous “move” that has no economic meaning and will very likely fire a false signal (or a false stop-out). **Adjusted close/OHLC data (back-adjusted for splits and bonuses) is required** for any backtest spanning a split event.
- **Dividends:** for indicator and signal-generation purposes, use prices adjusted for dividends is optional — it primarily matters for total-return accuracy, less for the shape of price action feeding a short-horizon technical rule. However, if the position-sizing

or P&L calculation is meant to represent actual investor return including dividends received while holding a position, dividend income must be added explicitly to the trade ledger — do not rely on adjusted-price P&L to implicitly capture it unless that convention is documented and consistent throughout.

- **Adjusted vs. unadjusted trade-off:** fully back-adjusted data is correct for signal generation and long-history indicator continuity, but back-adjusted historical prices do not represent prices that were tradeable on that historical date (they are theoretical, scaled backward from the present). This is fine for backtesting logic but must never be mixed with unadjusted data (e.g., an unadjusted stop level compared against an adjusted OHLC series) — pick one adjustment convention and hold it for every field in a given backtest run.

5.8 NSE/BSE-specific considerations

- NSE and BSE can show slightly different OHLC prints for the same symbol on the same day due to different participant flow; a multi-exchange or “best of both” price series must not be constructed by mixing bars from both exchanges within a single continuous series without a documented, consistent selection rule (e.g., “always NSE unless NSE data is missing”).
- Circuit filters (price bands) on Indian equities can prevent a stop or target from being filled even at the next available price, if the stock is locked at a circuit limit — the Section 9 intrabar policy and the Section 11 slippage model must account for the possibility that neither the stop nor the target price was actually reachable on a given day. This is a real, not theoretical, Indian-market execution risk and must not be quietly ignored.
- Do not assume T+1/T+2 settlement mechanics have any bearing on backtest signal timing — settlement affects when cash/shares move, not when a market order could be placed and filled. Keep the two concepts separate in the implementation.

No specific data vendor is assumed or recommended by this specification. Any example vendor name that may appear elsewhere in supporting material is illustrative only.

6 Indicator Specification

Ambiguity here is the single largest source of two implementations (“Python version” and “Java version”) producing different backtest results despite implementing “the same strategy.” Every formula below must be followed exactly.

6.1 EMA(50)

Smoothing factor:

$$\alpha = \frac{2}{N+1}, \quad N = 50 \Rightarrow \alpha = \frac{2}{51} \approx 0.03922$$

Recursive formula:

$$\text{EMA}[t] = \alpha \times \text{Close}[t] + (1 - \alpha) \times \text{EMA}[t - 1]$$

Initialization (required, not optional): EMA[0] (the seed value) must be the simple moving average (SMA) of the first N closes, not the first close price alone and not zero. Formally, for a series indexed from 0:

$$\text{EMA}[N - 1] = \text{SMA}(\text{Close}[0..N - 1]) = \frac{1}{N} \sum_{i=0}^{N-1} \text{Close}[i]$$

then apply the recursive formula for all $t \geq N$. Bars $t < N - 1$ have **no defined EMA value** — they are warm-up bars (Section 6), not zero, not the seed value repeated backward.

6.2 RSI(14) — Wilder’s method

Let $\Delta[t] = \text{Close}[t] - \text{Close}[t - 1]$.

$$\text{Gain}[t] = \max(\Delta[t], 0), \quad \text{Loss}[t] = \max(-\Delta[t], 0)$$

Initialization (using the first 14 deltas, i.e., bars 1..14 relative to bar 0):

$$\text{AvgGain}[14] = \frac{1}{14} \sum_{i=1}^{14} \text{Gain}[i], \quad \text{AvgLoss}[14] = \frac{1}{14} \sum_{i=1}^{14} \text{Loss}[i]$$

Wilder smoothing thereafter (for $t > 14$):

$$\begin{aligned} \text{AvgGain}[t] &= \frac{\text{AvgGain}[t - 1] \times 13 + \text{Gain}[t]}{14} \\ \text{AvgLoss}[t] &= \frac{\text{AvgLoss}[t - 1] \times 13 + \text{Loss}[t]}{14} \end{aligned}$$

RS and RSI:

$$RS[t] = \frac{\text{AvgGain}[t]}{\text{AvgLoss}[t]}, \quad \text{RSI}[t] = 100 - \frac{100}{1 + RS[t]}$$

Division-by-zero handling: if $\text{AvgLoss}[t] == 0$ (no down bars in the smoothing window — a strong, uninterrupted uptrend), RS is undefined (division by zero). The correct convention, matching the standard Wilder definition, is $\text{RSI}[t] = 100$ in this case, not an exception, not NaN, not zero. Implement this as an explicit branch, not by relying on a language’s floating-point inf behavior, because $100 / (1 + \text{inf})$ correctly evaluates toward 0 in IEEE 754 floating point (giving $\text{RSI} \rightarrow 100$) in some languages but this must not be left to accident — write the branch explicitly and test it (Section 28).

Wilder smoothing is not the same as an EMA with $\alpha = 2/15$. It is closer to an EMA with $\alpha = 1/14$. Do not substitute a standard EMA-based RSI implementation and assume it is equivalent — the numerical results diverge, especially in the first 30-50 bars after warm-up, which is exactly the region most examples/tests will exercise.

6.3 ATR(14) — Wilder’s method (the default; this is not SMA-ATR or EMA-ATR)

True Range:

$$\text{TR}[t] = \max \left(\text{High}[t] - \text{Low}[t], |\text{High}[t] - \text{Close}[t - 1]|, |\text{Low}[t] - \text{Close}[t - 1]| \right)$$

Close[t-1] requires a previous bar; TR[0] for the very first bar in the series is undefined (or, if unavoidable, High[0]-Low[0] with the gap terms omitted, but this first value should not be relied upon and is naturally superseded once warm-up completes).

Initialization:

$$\text{ATR}[14] = \frac{1}{14} \sum_{i=1}^{14} \text{TR}[i]$$

Wilder smoothing thereafter (for $t > 14$):

$$\text{ATR}[t] = \frac{\text{ATR}[t-1] \times 13 + \text{TR}[t]}{14}$$

This document explicitly disambiguates three commonly confused variants — implementers must use Wilder’s version (above) as the default for this strategy:

Variant	Smoothing	Use here?
SMA-ATR	Simple rolling mean of TR	No — too reactive to single outlier bars, not the historical convention
EMA-ATR	Standard EMA ($\alpha = 2/(N + 1)$) of TR	No — close to, but numerically distinct from, Wilder’s version
Wilder ATR	$\alpha = 1/N$ recursive smoothing (above)	Yes — the required default

Any deviation must be an explicit, documented, versioned configuration choice, not an implementation detail left to whichever library the developer happened to import.

7 Warm-Up Period

Definition: the number of leading bars for which one or more required indicators do not yet have a valid, fully-initialized value.

With default parameters:

Indicator	Bars needed before first valid value
EMA(50)	50
EMA(50) 5-bar slope	50 + 5 = 55
RSI(14)	14 (plus 1 for the first delta) = 15
ATR(14)	14 (plus 1 for the first TR) = 15
Combined strategy warm-up	max(55, 15, 15) = 55 bars

Rule: the system must not evaluate the entry condition (Section 3.3) on any bar $t < 55$ (with default parameters; this must be computed dynamically as EMA_period + slope_bars if parameters change, per Section 20). Signals on partially-initialized indicators are not just “less accurate” — they are computed from values that do not correspond to the specified formula at all (e.g., an EMA seeded from fewer than 50 closes is not EMA(50)), and any trade

generated from one is not a valid instance of the strategy under test. The IndicatorEngine (Section 7) must expose an explicit `is_ready` flag per bar, and the SignalEngine must refuse to evaluate entry logic when it is false — this should be a hard assertion in code, not a comment.

8 Backtest Engine Architecture

8.1 Component list and responsibilities

Component	Responsibility	Inputs	Outputs	Depends on
DataLoader	Read raw OHLCV from file/DB into a canonical in-memory series	File path / connection, symbol, date range	Ordered list of raw candle records	—
DataValidator	Apply Section 4.6 rules; reject/flag bad bars; detect duplicates/gaps	Raw candle series	Clean candle series + rejection log	DataLoader
IndicatorEngine	Compute EMA/RSI/ATR per Section 5; track warm-up/readiness	Clean candle series, indicator config	Per-bar indicator values + <code>is_ready</code> flag	DataValidator
Strategy	Evaluate Section 3 entry logic on ready bars; own stop/target formula	Candle + indicator series	Signal events (bar index, direction, stop, target)	IndicatorEngine
SignalEngine	Turn strategy signals into scheduled next-bar orders; enforce one-position-at-a-time	Signal events, current position state	Order intents	Strategy, PositionManager
ExecutionSimulator	Apply Section 3.4 (next-open fill), Section 9 (intraday policy), Section 11 (slippage)	Order intents, next bar's OHLC	Filled trade records (entry/exit)	SignalEngine
PositionManager	Track the single open position (if any) per symbol; check stop/target each bar	Filled entries, each new bar's High/Low	Exit triggers	ExecutionSimulator

Component	Responsibility	Inputs	Outputs	Depends on
Portfolio	Track cash, equity, realized/unrealized P&L across time	Filled trades, mark-to-market prices	Equity curve series	PositionManager, CostModel
TradeLedger	Persist every completed trade with full field set (Section 14)	Completed trades	Structured trade log (e.g., DataFrame/DB table)	Portfolio
CostModel	Compute broker-age/STT/GST/stamp duty/other charges per trade	Trade value, side, configured rate schedule	Total cost per leg	Configuration
RiskManager	Enforce Section 37 system-level limits (max risk/trade, max daily loss, etc.)	Portfolio state, proposed order	Approve/veto order	Portfolio, SignalEngine
PerformanceAnalyzer	Compute Section 15 metrics from the trade ledger and equity curve	TradeLedger, equity curve	Metrics dictionary/object	TradeLedger, Portfolio
ReportGenerator	Render Section 40 output report	Metrics, charts data, configuration	PDF/HTML/console report	PerformanceAnalyzer

8.2 Key edge cases per component

- DataLoader: empty file, symbol not found, malformed timestamp formats, mixed time-zones across rows.
- DataValidator: entire dataset fails validation (must not silently return an empty series and let downstream components proceed as if nothing happened — must raise/halt).
- IndicatorEngine: parameter change mid-series (should not be supported within a single run — indicators must be computed once per fixed parameter set for the whole series).
- Strategy: two signal conditions becoming true on the same bar after a data gap is filled (the slope condition briefly true only because of an interpolation artifact) — this is exactly why Section 4.4 forbids interpolating price data.
- ExecutionSimulator: order scheduled for next-bar entry, but the series ends before the next bar exists (drop the order, log it, do not fabricate a fill).
- PositionManager: both stop and target conditions satisfied within the same bar (Section 9 — this is not this component’s decision to make ad hoc; it must call the configured intrabar policy).
- Portfolio: position size calculates to zero shares (Section 12.3) — no trade should be recorded, not a zero-quantity trade.
- CostModel: negative computed cost due to a misconfigured rate (must validate configuration at startup, not at trade time).
- RiskManager: a valid signal arrives while max_positions is already reached — correctly reject without corrupting the pending-signal queue (the signal is simply not actionable that day, it is not queued for a future day either, since that would violate Section 3.4’s timing).

9 Backtest Event Flow

9.1 Per-bar sequence

For each new candle t (in chronological order):

1. Load candle t
2. Validate candle t (DataValidator)
3. Update indicators using candle t (IndicatorEngine)
4. IF a position is currently open:
 - a. Check exit conditions using candle t 's High/Low against the position's Stop/Target (PositionManager, Section 9 policy)
 - b. IF exit triggered: execute exit at the appropriate price, apply costs/slippage, close position, write trade to TradeLedger
5. IF indicators are_ready AND no position is open AND no order is already pending for tomorrow's open:
 - a. Evaluate entry condition using candle t (Strategy, Section 3.3)
 - b. IF signal fires: compute Stop/Target from candle t 's ATR (Section 3.5–3.6); schedule an entry order for candle $t+1$'s open
6. IF an order was scheduled on the PREVIOUS bar ($t-1$) for entry at this bar's open:
 - a. Execute entry at candle t 's Open, apply slippage/costs (Section 11), open position
7. Update Portfolio mark-to-market using candle t 's Close
8. Record equity/state snapshot for candle t

9.2 Look-ahead-bias analysis of this ordering

The critical property of the sequence above is that **step 5's decision (evaluate signal) and step 6's action (execute a previously-scheduled entry) never use the same bar's data for both signal and fill**. A signal computed from bar t 's close is only ever acted on at bar $t+1$'s open, which is realized in the ordering by “schedule at step 5 of bar t ” → “execute at step 6 of bar $t+1$.” Step 4 (checking existing-position exits) is allowed to use bar t 's High/Low because that is the position's *own* bar being evaluated for an intrabar stop/target touch — this is standard and not look-ahead, provided the intrabar ambiguity (Section 9) is handled honestly rather than assumed favorably.

A naive alternative ordering — evaluate the signal and fill it within the same step, using the same bar's close as both signal and fill price — is explicitly rejected (Section 3.4) because it fills at a price that could not have been known before the signal existed.

9.3 Recommended final ordering

The 8-step sequence in 8.1 is the specified, final ordering for V1.0. Two implementation notes:

- Step 4 (exit check) must run **before** step 5 (new entry check) on the same bar, so that a position closed intraday on bar t does not block a fresh signal from being evaluated on that same bar (the “no position open” gate in step 5 must reflect the *post-exit* state).
- Step 6 (execute previously-scheduled entry) can be implemented either before or after step 4/5 of the *same* bar without changing correctness, since it operates on an order scheduled a full bar earlier and only touches that bar's Open — but it must never be

reordered to run before step 3 (indicator update), since downstream reporting assumes indicators are current as of the bar being processed.

10 Intrabar Ambiguity

10.1 The problem

OHLC data collapses everything that happened inside a bar into four numbers. If, on a single bar, **both** of the following are true:

$$\text{High}[t] \geq \text{Target} \quad \text{AND} \quad \text{Low}[t] \leq \text{Stop}$$

the daily bar alone cannot tell you whether price touched the target first (in which case the trade should have exited as a winner) or the stop first (a loser), or oscillated between the two multiple times before the bar closed. This is not a rare corner case in a mean-reverting-adjacent risk/reward setup like this one (tight stop, wider target, both plausibly inside one volatile day's range) — it must be handled explicitly, not hoped away.

10.2 Candidate solutions

1. **Conservative assumption:** whenever both are touched in the same bar, assume the worse outcome for the trader — the stop was hit (first). This understates realized profit factor and win rate relative to reality but never overstates them, which is the safer direction to be wrong in for research purposes.
2. **Lower-timeframe reconstruction:** fetch intraday (e.g., 1-minute or 5-minute) data for the ambiguous days only, and replay it to see which level was actually touched first. This is accurate but expensive — it requires a second, higher-resolution data source and only for the subset of bars where ambiguity occurs.
3. **Tick-level simulation:** full order-book/tick replay. This is the most accurate possible answer but is a materially larger engineering effort disproportionate to a daily-bar swing strategy, and tick data for the full backtest universe/history is often not practically available at reasonable cost for Indian equities research use.

10.3 Default policy for V1.0 (stated explicitly as an assumption, not hidden)

Default: Conservative assumption (Option 1) — stop-loss takes priority over take-profit within an ambiguous bar.

This is a deliberate, documented, conservative bias. Every backtest report generated by this system must state this assumption explicitly in its methodology section (Section 40), and the TradeLedger must flag every trade where both stop and target were touched on the same bar (`ambiguous_exit = true`) so that the frequency of this situation — and therefore the report's sensitivity to this assumption — is itself measurable and disclosed, not buried.

Optional (V1.2+, Section 43): Option 2 (lower-timeframe reconstruction) as a configurable, opt-in verification pass specifically for ambiguous-bar trades, to quantify how much the conservative default assumption is costing the reported performance versus a more accurate resolution.

11 Transaction Cost Model

11.1 Requirement

The CostModel must support, as independently configurable line items:

- Brokerage
- Exchange transaction charges
- Securities Transaction Tax (STT)
- Goods and Services Tax (GST)
- SEBI turnover charges
- Stamp duty
- Slippage (modeled separately — Section 11 — but it flows into the same net-P&L calculation)
- Other/custom configurable charges (a catch-all line item for anything not enumerated above)

11.2 Explicit non-requirement

Do not hardcode current Indian brokerage or statutory rates into the codebase as constants. Rates change (GST rate changes, SEBI/exchange fee revisions, broker-specific brokerage plans), and hardcoding them silently makes every historical backtest report wrong the day the law or the broker's plan changes, with no error raised anywhere. All rates must live in the configuration (Section 38), loaded at run time, and every generated report must print the exact rate schedule used for that run (Section 47, Reproducibility) so a stale-configuration mistake is at least visible after the fact.

11.3 Illustrative example configuration

The structure below reflects the general shape of NSE cash-market delivery trade charges as commonly published by brokers (transaction charges on both legs, STT on both legs for delivery trades, stamp duty on the buy leg, GST layered on brokerage + exchange + SEBI charges, SEBI charges per-crore). **The specific numeric rates are illustrative examples only, sourced from broker/exchange fee-schedule pages current as of research time, and must be re-verified against the current, authoritative NSE/SEBI/broker circulars before being used in any real analysis** — do not treat the numbers below as production values.

```
cost_model:
  brokerage:
    type: percentage          # or flat_per_order, or greater_of
    value: 0.0                # e.g., many discount brokers: 0 or flat Rs per order
  exchange_transaction_charge_pct: 0.00297  # illustrative, both legs
  stt_pct_delivery: 0.10      # illustrative, both buy and sell legs
  sebi_charges_per_crore: 10  # illustrative, both legs
  stamp_duty_pct_buy_leg: 0.015 # illustrative, buy leg only
  gst_pct_on_brokerage_and_exchange_and_sebi: 18.0 # illustrative
  other_charges_flat: 0.0
```

11.4 Why ignoring costs distorts results, especially for short-horizon strategies

A strategy with an average holding period of a few days and a modest per-trade edge can have its entire statistical edge consumed by round-trip costs that are individually small (fractions

of a percent) but apply on both entry and exit, every single trade, with certainty — unlike the trading edge itself, which is uncertain. A backtest that omits costs is not “close enough” — it is testing a different, easier, non-existent strategy: one that trades for free. Any performance figure in this document’s output report (Section 40) must be reported net of the configured cost model; gross-of-cost figures may be shown alongside for diagnostic purposes but must never be the headline number.

12 Slippage Model

12.1 Supported types

Type	Definition	Example
Fixed	A constant absolute price offset per trade	₹0.50 per share, always against the trader
Percentage	A % of the fill price	0.05% of fill price
Basis points	Same as percentage, expressed in bps	5 bps of fill price

Exactly one slippage type is active per backtest run (configurable), applied consistently to entries and exits unless separately configured per leg.

12.2 Where slippage applies

- **Entry:** the simulated fill price is the next bar’s open (Section 3.4), *adjusted against the trader* by the configured slippage — for a long entry, $\text{fill} = \text{Open}[t+1] \times (1 + \text{slippage_pct})$ (worse, i.e., higher, than the raw open).
- **Stop exit:** $\text{fill} = \text{Stop} \times (1 - \text{slippage_pct})$ (worse, i.e., lower, than the theoretical stop level) — stops are disproportionately prone to adverse slippage in fast-moving or gapped markets, and the model must not pretend stops fill at the exact stop price.
- **Target exit:** $\text{fill} = \text{Target} \times (1 - \text{slippage_pct})$ (also adjusted against the trader by convention, though target fills in liquid names are typically less adversely affected than stop fills — a more granular, direction-specific slippage config is a valid future enhancement, Section 43).
- **P&L:** computed from realized fill prices (post-slippage), never from the theoretical signal/stop/target prices. The theoretical prices are retained in the TradeLedger (Section 14) purely for auditability/comparison, clearly labeled as pre-slippage reference values.

13 Position Sizing

13.1 Fixed quantity

$$\text{Quantity} = Q_{\text{fixed}} \quad (\text{e.g., 100 shares, configured per symbol or globally})$$

Simple, but ignores both account size and per-trade risk variability driven by ATR — different trades will carry very different rupee risk under this method, purely as a side effect of volatility at signal time.

13.2 Fixed rupee risk (recommended default)

$$\text{Risk per share} = \text{Entry} - \text{Stop} = 1.5 \times \text{ATR}_{14}[t]$$

$$\text{Position Size} = \left\lfloor \frac{\text{Maximum Risk}}{\text{Risk per share}} \right\rfloor \quad (\text{both in rupees})$$

Example: maximum risk = ₹1,000, risk per share = ₹5.00 → raw size = 200 shares. The floor function enforces integer share rounding (fractional shares are not tradeable on NSE/BSE cash equity).

13.3 Zero-size edge case

If Maximum Risk / Risk per share < 1 (i.e., the ATR-based stop distance alone exceeds the maximum rupee risk budget for even a single share), the computed position size floors to zero. **The system must not silently skip the trade with no record** — it must log a `signal_rejected: position_size_zero` event, distinct from a normal “no signal” bar, since it represents a valid entry signal that could not be sized under the current risk configuration. This distinction matters for correctly interpreting “number of signals” vs. “number of trades taken” in the final report.

14 Capital Model

Concept	Definition
Starting capital	Configured initial cash balance for the backtest run
Available cash	Starting capital, adjusted for realized P&L, minus capital currently committed to the open position
Position value	Quantity × current mark price for the open position, if any
Realized P&L	Sum of net P&L (post-cost, post-slippage) from all closed trades
Unrealized P&L	(Current mark price - Entry price) × Quantity for the open position, marked at each bar’s close
Equity	Available cash + Position value (equivalently: Starting capital + Realized P&L + Unrealized P&L)
Margin	Not modeled in V1.0 (see below)

Base model assumption: cash equity, fully paid, no margin/leverage, unless a future configuration explicitly enables margin (out of scope for V1.0; flagged in Section 43 as a V2.0+ candidate). Under this assumption, a position can only be sized up to available cash — if the fixed-rupee-risk sizing (Section 12.2) computes a share count whose total value exceeds available cash, the system must cap the position at the maximum affordable quantity and log this as a `capital_constrained` event, not silently borrow phantom capital to fill the full computed size.

15 Trade Ledger

Every completed trade is a single row with, at minimum, the following fields:

Field	Type	Notes
trade_id	string/int	Unique per trade
symbol	string	
signal_timestamp	datetime	Bar t where the entry condition fired
entry_timestamp	datetime	Bar $t+1$, when the order was actually filled
exit_timestamp	datetime	Bar on which stop/target/end-of-data triggered
entry_price	float	Post-slippage fill price
exit_price	float	Post-slippage fill price
stop_price	float	Theoretical stop level (pre-slippage), fixed at signal time
target_price	float	Theoretical target level (pre-slippage), fixed at signal time
atr_at_signal	float	ATR14[t] used to size stop/target
risk_per_share	float	$\text{entry_price} - \text{stop_price}$ (using theoretical or realized entry, documented consistently)
quantity	int	Shares
gross_pnl	float	Before costs
total_costs	float	Sum of all CostModel line items, both legs
net_pnl	float	$\text{gross_pnl} - \text{total_costs}$
r_multiple	float	$\text{net_pnl} / (\text{risk_per_share} \times \text{quantity})$
holding_period_bars	int	$\text{exit_timestamp} - \text{entry_timestamp}$ in bars
exit_reason	enum	STOP, TARGET, END_OF_DATA
ambiguous_exit	bool	True if both stop and target were touchable in the exit bar (Section 9)

16 Performance Metrics

Every metric below must be computed from the post-cost, post-slippage TradeLedger and equity curve, never from theoretical/pre-cost values, unless explicitly labeled “gross” for diagnostic comparison.

Metric	Formula	Limitation
Total trades	Count of closed trades	Excludes signal_rejected events (Section 12.3) — report those separately
Win rate	Wins / Total trades	Meaningless without average win/loss size alongside it
Average win / loss	Mean net_pnl of winners / losers	Sensitive to outliers; report median alongside
Reward/risk (realized)	Average win ÷ Average loss	Realized, not the theoretical 2R — differs due to costs, slippage, and Section 9 ambiguity resolution
Expectancy	(Win rate × Avg win) + (Loss rate × Avg loss)	Per-trade average edge in currency terms; loss is signed negative
Profit factor	Gross profit ÷ Gross loss	Undefined/infinite if there are zero losing trades — report as such, not as a large finite number
Gross profit / loss	Sum of positive / negative net_pnl	—
Net P&L	Sum of all net_pnl	Headline currency return
Maximum drawdown	Max peak-to-trough decline in the equity curve	A single historical path; says nothing about future worst case (Section 22 addresses this)
Drawdown duration	Longest time from a peak to full recovery	Can be undefined if the backtest ends still in drawdown — must be flagged, not silently reported as the partial value
CAGR	$\left(\frac{\text{Equity}_{\text{end}}}{\text{Equity}_{\text{start}}}\right)^{1/\text{years}} - 1$	Only meaningful over a reasonably long period; misleading over short backtests
Sharpe ratio	$\frac{\text{mean}(r) - r_f}{\frac{\text{std}(r)}{\sqrt{\text{periods per year}}}} \times$	Assumes returns are roughly normal/stationary; a trend-following strategy's return stream typically is not
Sortino ratio	Same as Sharpe but denominator uses downside deviation only	Better suited to asymmetric return distributions like this strategy's, still a single-history estimate
Calmar ratio	CAGR ÷ Max drawdown	Very sensitive to the single worst historical drawdown episode
Average / median holding period	Mean / median of holding_periodBars	—

Metric	Formula	Limitation
Max consecutive wins / losses	Longest streak in either direction	Useful for psychological/risk-of-ruin framing, not a predictive statistic
Exposure	% of trading days with an open position	Low exposure means capital sits idle — an opportunity cost not captured by per-trade metrics alone
Turnover	Sum of traded value ÷ average capital, per period	Relevant to cost-sensitivity and capacity discussions

General limitation applying to every metric in this section: all are computed on one historical realization of price data and one historical realization of the strategy's own trade sequence. None of them, individually, indicates what will happen going forward. Section 22 (Monte Carlo) exists specifically to turn a single point estimate into a distribution.

17 Equity Curve and Visualizations

Visualization	What it must communicate
Equity curve	Cumulative account value over time, net of costs; overlay drawdown shading
Drawdown curve	Percentage decline from running peak equity, over time — makes clustering/duration of drawdowns visible in a way the single “max drawdown” number cannot
Monthly returns (table/heatmap)	Return by calendar month, to visually surface seasonality or regime concentration
Trade P&L distribution (histogram)	Shape of individual trade outcomes — is the edge broad-based or a few large winners carrying many small losers?
R-multiple distribution (histogram)	Same question, normalized to risk taken per trade, comparable across trades with different ATR at signal time
Win/loss distribution (bar or box plot)	Separately visualize the winning-trade and losing-trade size distributions, since averaging them together (as in “average trade”) hides asymmetry

18 Benchmarking

18.1 Required comparisons

- **NIFTY 50** (or the relevant broad index for the universe traded) — buy-and-hold, same period.
- **Relevant sector index**, if the traded symbol(s) belong to a specific sector, to separate “did the strategy beat the market” from “did the strategy just happen to trade a stock that was itself a strong performer.”

- **Buy-and-hold of the same underlying symbol** over the identical period — this is the most important comparison for a single-stock backtest, because it directly isolates the value added (or destroyed) by the entry/exit timing logic versus simply holding the stock.

18.2 Required benchmark metrics (computed identically to the strategy's own)

Return, volatility (annualized standard deviation of returns), maximum drawdown, and a risk-adjusted measure (Sharpe or Sortino) — computed on the benchmark using the same period, same cost assumptions where applicable (buy-and-hold has minimal turnover cost, but it is not exactly zero either — a single entry and, if closed, a single exit).

18.3 Why benchmark selection matters, and what not to claim

A strategy that returns more than a buy-and-hold of the same stock but takes on similar or higher drawdown and volatility has not obviously “beaten” buy-and-hold on a risk-adjusted basis — report both the raw return comparison and the risk-adjusted comparison side by side, and do not headline the raw return number alone. Higher raw return with materially lower drawdown is a meaningfully different (and stronger) claim than higher raw return with similar or higher drawdown; the report template (Section 40) must make this distinction explicit rather than let a single “beat the benchmark by X%” line imply more than it supports.

19 Research Methodology

19.1 Three-way data split

Period	Purpose	Rule
In-sample	Initial strategy development and rule-writing	Where the strategy rules in Section 3 were conceived/reasoned about
Validation	Parameter selection checks (Section 20 sensitivity sweeps)	Used repeatedly during development, but tracked — every sweep run against it is logged
Out-of-sample (OOS)	Final, one-time check	Touched exactly once , only after the strategy and its parameters are fully frozen

19.2 Why repeated testing on the same data causes overfitting

Every time a parameter is adjusted and re-tested against the same historical window in search of a better metric, the researcher is (even unintentionally) selecting for whatever random noise in that specific historical window happens to favor the current parameter choice. After enough iterations, the strategy is optimized to the noise of that one dataset, not to a real, repeatable market effect — and it will typically underperform, sometimes dramatically, on data it has never seen. The out-of-sample period’s entire value comes from never having been used this way; using it more than once for a real decision (not just a final report) destroys that value permanently, because now it too has been “peeked at.”

Practical rule enforced by process, not code: log the date and result of every OOS evaluation (Section 47). If OOS is evaluated more than once during development with parameter

changes made in between, treat all “OOS” claims for that project as compromised in-sample results and disclose this explicitly in the report.

20 Walk-Forward Testing

20.1 Framework

```
Window 1: [ Train ][ Test ]
Window 2:      [ Train ][ Test ]
Window 3:      [ Train ][ Test ]
Window 4:      [ Train ][ Test ]
```

Each “Test” window’s result is recorded independently; the final walk-forward performance is the **concatenation of only the Test segments** across all windows — never the Train segments, and never a single combined Train+Test backtest.

20.2 Parameters to define

Parameter	Description	Default (illustrative, tune to available history)
Training window	Length of history used for parameter selection	e.g., 2 years
Testing window	Length of subsequent out-of-sample segment	e.g., 6 months
Window type	Rolling (fixed length, slides forward) vs. expanding (start fixed, end grows)	Rolling is the more conservative default — it does not let the model “remember” arbitrarily distant history indefinitely
Parameter selection	How parameters are chosen within each Train window	Sensitivity-informed selection (Section 20), not blind
Refit frequency	How often the model is re-optimized	grid-search-for-best-Sharpe Matches the testing window length (i.e., refit at the start of every new Test window)
Aggregation	How per-window Test results are combined into an overall statistic	Concatenate trade ledgers from all Test windows; compute Section 15 metrics on the combined ledger, and also report per-window metrics to check for consistency across windows

20.3 What a good walk-forward result looks like

Reasonably consistent sign and rough magnitude of edge across most Test windows — not necessarily identical performance in every window (regimes differ), but not a strategy that is wildly profitable in some windows and catastrophically negative in others, which would indicate the “edge” is really regime-specific luck rather than a stable, re-discoverable effect.

21 Parameter Sensitivity

21.1 Base parameters

EMA period	= 50
EMA slope window	= 5 bars
RSI period	= 14
RSI reclaim level	= 40
ATR period	= 14
Stop distance	= 1.5 × ATR
Reward:risk	= 2.0 (Target = 2R)

21.2 Sensitivity sweep ranges

Parameter	Values to test
EMA period	20, 30, 50, 75, 100
EMA slope window	3, 5, 8, 10
RSI period	(hold at 14 unless specifically investigating; RSI reclaim level is the primary lever)
RSI reclaim level	30, 35, 40, 45
ATR stop multiple	1.0, 1.25, 1.5, 1.75, 2.0
Reward:risk	1.5, 2.0, 2.5, 3.0

21.3 What the sweep must NOT be used for

The sweep must never be used to hand-pick the single combination with the highest historical return or Sharpe ratio and declare that the new “optimized” parameter set. That is overfitting by definition — it selects for whichever combination best matched the noise of the specific historical sample, with no reason to expect that specific combination to be special going forward.

21.4 What the sweep IS for: robustness and parameter-stability assessment

Plot or tabulate the chosen metric (e.g., expectancy, profit factor) as a smooth surface/curve across the parameter range. A **robust** result looks like a plateau — performance is broadly similar across a wide neighborhood of the chosen base parameters (e.g., RSI level 35–45 all show a positive, broadly similar edge). A **fragile, likely-overfit** result looks like a narrow spike — one specific parameter combination (e.g., RSI level exactly 41) looks great while its immediate neighbors (39, 43) look mediocre or negative. Only the plateau pattern gives any justification for using the base parameters with some confidence; a spike pattern is itself a failure signal (Section 45) regardless of how good the spike’s own number looks.

22 Overfitting Controls

Control	What it catches
Out-of-sample testing (Section 18)	Whether the strategy generalizes beyond the data it was designed on at all

Control	What it catches
Walk-forward analysis (Section 19)	Whether an edge is stable across multiple, sequential, non-overlapping time periods
Parameter sensitivity (Section 20)	Whether the result depends on a narrow, coincidental parameter choice
Monte Carlo analysis (Section 22)	Whether the <i>specific sequence</i> of historical trades/returns was a lucky draw
Trade-order randomization	Whether the P&L path (not just the total) depended on trades happening in this exact historical order
Bootstrap analysis	Resampling trades with replacement to build a confidence interval around metrics like expectancy, rather than reporting a single point estimate
Stress testing	Deliberately adverse scenarios (e.g., a hypothetical extended drawdown period, a liquidity shock) beyond what happened to occur historically
Slippage sensitivity	Whether the edge survives materially worse slippage assumptions than the base case
Cost sensitivity	Whether the edge survives a higher brokerage/tax regime than currently modeled

Each of these targets a *different* way a backtest can lie to its author; passing one does not substitute for the others.

23 Monte Carlo Analysis

23.1 What to randomize

- **Trade sequence** — reshuffle the order of the historical trade ledger's `net_pnl` values (with replacement, for bootstrapping, or without, for permutation) to build a distribution of possible equity-curve *paths* consistent with the same set of trade outcomes.
- **Returns** — resample per-trade returns (as R-multiples) rather than absolute P&L, to make the exercise position-sizing-independent, then re-apply a fixed position-sizing rule to each resampled path.
- **Slippage** — draw slippage per trade from a distribution around the configured base assumption, rather than a fixed constant, to see how sensitive outcomes are to slippage variability, not just its average.
- **Execution assumptions** — e.g., randomly resolve Section 9's stop/target ambiguity in a subset of ambiguous bars using the *less conservative* outcome, to bound how much the conservative default (Section 9.3) might be costing the reported numbers, rather than just the direction of the bias.

23.2 Outputs

- Distribution of possible maximum drawdowns (report percentiles, e.g., 5th/50th/95th, not just the single historical worst case).
- Approximate probability of ruin at the configured position sizing (fraction of simulated paths breaching a defined capital-loss threshold).

- Equity distribution at the end of the simulated horizon (spread, not just the mean).
- Worst-case simulated sequences, for qualitative review (e.g., “what did the 5 worst simulated paths actually look like”).

23.3 Explicit limitation

Monte Carlo resampling of historical trades does not prove, or even estimate, future profitability. It only characterizes how much variability existed *within the specific set of historical outcomes already observed*, under different possible orderings and perturbations. If the underlying edge itself disappears going forward (Section 2.3), no amount of Monte Carlo analysis on the old trade set will detect that — it is a robustness check on a known sample, not a forecast.

24 Python Implementation

24.1 Recommended project structure

```
quant_strategy/
├── data/
│   ├── loader.py           # DataLoader
│   └── validator.py        # DataValidator
├── indicators/
│   ├── ema.py
│   ├── rsi.py
│   └── atr.py
├── strategy/
│   └── trend_pullback.py    # Strategy (Section 3 rules)
├── backtest/
│   ├── signal_engine.py
│   ├── execution.py         # ExecutionSimulator, intrabar policy
│   ├── position.py          # PositionManager
│   └── engine.py            # Orchestrates the Section 8 event loop
├── execution/
│   ├── cost_model.py
│   └── slippage.py
├── portfolio/
│   ├── portfolio.py
│   └── ledger.py            # TradeLedger
├── metrics/
│   ├── performance.py      # Section 15
│   └── monte_carlo.py      # Section 22
├── visualization/
│   └── charts.py            # Section 16
```

```

├── reports/
│   └── report_generator.py  # Section 40
├── tests/
│   └── ...                  # Section 28
├── config/
│   └── default_config.yaml  # Section 38
└── main.py

```

Dependencies: pandas, numpy, matplotlib, pytest. vectorbt and backtrader are mentioned only as *alternatives a team might consider instead of this custom engine* — they are not dependencies of this design. The core backtest loop is written in plain pandas/NumPy so that the Section 8 event flow is directly visible in the code, not hidden inside a third-party framework's internal event system.

24.2 Configuration schema (Python side, mirrors Section 38 YAML)

```

from dataclasses import dataclass, field

@dataclass
class StrategyConfig:
    ema_period: int = 50
    slope_bars: int = 5
    rsi_period: int = 14
    rsi_reclaim_level: float = 40.0
    atr_period: int = 14
    stop_atr_multiple: float = 1.5
    reward_risk_multiple: float = 2.0

@dataclass
class ExecutionConfig:
    entry_timing: str = "next_open"      # only supported value in V1.0
    slippage_type: str = "percentage"     # "fixed" | "percentage" | "bps"
    slippage_value: float = 0.05         # meaning depends on slippage_type
    intrabar_policy: str = "conservative" # Section 9.3

@dataclass
class RiskConfig:
    max_risk_per_trade_rupees: float = 1000.0
    max_positions: int = 1
    max_daily_loss_rupees: float = 5000.0

@dataclass
class BacktestConfig:
    strategy: StrategyConfig = field(default_factory=StrategyConfig)
    execution: ExecutionConfig = field(default_factory=ExecutionConfig)
    risk: RiskConfig = field(default_factory=RiskConfig)
    starting_capital: float = 500_000.0

```

24.3 Example execution flow

```
from data.loader import DataLoader
from data.validator import DataValidator
from indicators import ema, rsi, atr
from strategy.trend_pullback import TrendPullbackStrategy
from backtest.engine import BacktestEngine
from metrics.performance import PerformanceAnalyzer
from reports.report_generator import ReportGenerator
from config import BacktestConfig

def run_backtest(symbol: str, start: str, end: str, config: BacktestConfig):
    raw = DataLoader().load(symbol, start, end)
    clean, rejections = DataValidator().validate(raw)

    clean["ema50"] = ema.wilder_style_ema(clean["close"], config.strategy.ema_period)
    clean["rsi14"] = rsi.wilder_rsi(clean["close"], config.strategy.rsi_period)
    clean["atr14"] = atr.wilder_atr(clean, config.strategy.atr_period)

    strategy = TrendPullbackStrategy(config.strategy)
    engine = BacktestEngine(strategy=strategy, config=config)
    ledger, equity_curve = engine.run(clean)

    metrics = PerformanceAnalyzer().compute(ledger, equity_curve, config.starting_capital)
    ReportGenerator().build(metrics, ledger, equity_curve, config, rejections)
    return ledger, metrics
```

25 Python Code Requirements — Representative Implementations

25.1 EMA (Section 5.1)

```
import numpy as np
import pandas as pd

def wilder_style_ema(close: pd.Series, period: int) -> pd.Series:
    """Standard EMA with  $\alpha = 2/(period+1)$ , seeded with the SMA
    of the first `period` closes. Returns NaN for all bars before
    the seed is available (Section 6, warm-up)."""
    if len(close) < period:
        return pd.Series(np.nan, index=close.index)

    alpha = 2.0 / (period + 1)
    ema_values = pd.Series(np.nan, index=close.index)

    seed = close.iloc[:period].mean()
    ema_values.iloc[period - 1] = seed

    for i in range(period, len(close)):
        prev = ema_values.iloc[i - 1]
        ema_values.iloc[i] = alpha * close.iloc[i] + (1 - alpha) * prev
```

```
return ema_values
```

25.2 RSI (Section 5.2), Wilder smoothing

```
def wilder_rsi(close: pd.Series, period: int = 14) -> pd.Series:
    delta = close.diff()
    gain = delta.clip(lower=0.0)
    loss = (-delta).clip(lower=0.0)

    avg_gain = pd.Series(np.nan, index=close.index)
    avg_loss = pd.Series(np.nan, index=close.index)

    if len(close) <= period:
        return pd.Series(np.nan, index=close.index)

    avg_gain.iloc[period] = gain.iloc[1:period + 1].mean()
    avg_loss.iloc[period] = loss.iloc[1:period + 1].mean()

    for i in range(period + 1, len(close)):
        avg_gain.iloc[i] = (avg_gain.iloc[i - 1] * (period - 1) + gain.iloc[i]) / period
        avg_loss.iloc[i] = (avg_loss.iloc[i - 1] * (period - 1) + loss.iloc[i]) / period

    rsi = pd.Series(np.nan, index=close.index)
    for i in range(period, len(close)):
        if avg_loss.iloc[i] == 0:
            rsi.iloc[i] = 100.0          # explicit branch - Section 5.2
        else:
            rs = avg_gain.iloc[i] / avg_loss.iloc[i]
            rsi.iloc[i] = 100.0 - (100.0 / (1.0 + rs))

    return rsi
```

25.3 ATR (Section 5.3), Wilder smoothing

```
def wilder_atr(candles: pd.DataFrame, period: int = 14) -> pd.Series:
    high, low, close = candles["high"], candles["low"], candles["close"]
    prev_close = close.shift(1)

    tr = pd.concat([
        high - low,
        (high - prev_close).abs(),
        (low - prev_close).abs(),
    ], axis=1).max(axis=1)

    atr = pd.Series(np.nan, index=candles.index)
    if len(candles) <= period:
        return atr

    atr.iloc[period] = tr.iloc[1:period + 1].mean()
    for i in range(period + 1, len(candles)):
        atr.iloc[i] = (atr.iloc[i - 1] * (period - 1) + tr.iloc[i]) / period
```

```
return atr
```

25.4 Signal generation (Section 3.3)

```
def generate_signals(df: pd.DataFrame, cfg) -> pd.Series:
    ema_now = df["ema50"]
    ema_prior = df["ema50"].shift(cfg.slope_bars)
    trend_ok = (ema_now > ema_prior) & (df["close"] > ema_now)

    rsi_now = df["rsi14"]
    rsi_prior = df["rsi14"].shift(1)
    reclaim = (rsi_prior < cfg.rsi_reclaim_level) & (rsi_now >= cfg.rsi_reclaim_level)

    is_ready = ema_now.notna() & ema_prior.notna() & rsi_now.notna() & rsi_prior.notna()

    return trend_ok & reclaim & is_ready
```

25.5 Position sizing (Section 12.2)

```
import math

def size_position(entry_price: float, stop_price: float,
                  max_risk_rupees: float, available_cash: float) -> int:
    risk_per_share = entry_price - stop_price
    if risk_per_share <= 0:
        return 0
    raw_qty = math.floor(max_risk_rupees / risk_per_share)
    max_affordable_qty = math.floor(available_cash / entry_price)
    return max(0, min(raw_qty, max_affordable_qty))
```

25.6 Trade execution / stop-target handling (Sections 3.4-3.7, 9)

```
from dataclasses import dataclass
from enum import Enum

class ExitReason(Enum):
    STOP = "STOP"
    TARGET = "TARGET"
    END_OF_DATA = "END_OF_DATA"

@dataclass
class OpenPosition:
    entry_price: float
    stop_price: float
    target_price: float
    quantity: int
    entry_timestamp: object

def check_exit(position: OpenPosition, bar_high: float, bar_low: float,
               intrabar_policy: str = "conservative"):
```

```

hit_stop = bar_low <= position.stop_price
hit_target = bar_high >= position.target_price

if hit_stop and hit_target:
    # Section 9.3 default: stop wins on ambiguity
    if intrabar_policy == "conservative":
        return ExitReason.STOP, position.stop_price, True
    raise NotImplementedError("Only 'conservative' policy is in V1.0 scope")
if hit_stop:
    return ExitReason.STOP, position.stop_price, False
if hit_target:
    return ExitReason.TARGET, position.target_price, False
return None, None, False

```

25.7 Cost calculation (Section 10)

```

def compute_trade_costs(trade_value_buy: float, trade_value_sell: float, cfg) -> float:
    def leg_cost(value, is_buy_leg: bool) -> float:
        brokerage = value * cfg.brokerage_pct if cfg.brokerage_type == "percentage" else cfg.br
        exch = value * cfg.exchange_transaction_charge_pct / 100
        stt = value * cfg.stt_pct_delivery / 100
        sebi = value * cfg.sebi_charges_per_crore / 1e7
        stamp = value * cfg.stamp_duty_pct_buy_leg / 100 if is_buy_leg else 0.0
        gst_base = brokerage + exch + sebi
        gst = gst_base * cfg.gst_pct / 100
        return brokerage + exch + stt + sebi + stamp + gst

    return leg_cost(trade_value_buy, True) + leg_cost(trade_value_sell, False)

```

25.8 Performance metrics (Section 15, representative subset)

```

def compute_core_metrics(ledger: pd.DataFrame) -> dict:
    wins = ledger[ledger["net_pnl"] > 0]
    losses = ledger[ledger["net_pnl"] <= 0]

    gross_profit = wins["net_pnl"].sum()
    gross_loss = losses["net_pnl"].sum() # negative

    win_rate = len(wins) / len(ledger) if len(ledger) else float("nan")
    avg_win = wins["net_pnl"].mean() if len(wins) else 0.0
    avg_loss = losses["net_pnl"].mean() if len(losses) else 0.0

    profit_factor = (gross_profit / abs(gross_loss)) if gross_loss != 0 else float("inf")
    expectancy = win_rate * avg_win + (1 - win_rate) * avg_loss

    return {
        "total_trades": len(ledger),
        "win_rate": win_rate,
        "avg_win": avg_win,
        "avg_loss": avg_loss,
        "profit_factor": profit_factor,
    }

```

```
"expectancy": expectancy,  
"gross_profit": gross_profit,  
"gross_loss": gross_loss,  
"net_pnl": ledger["net_pnl"].sum(),  
}
```

All snippets above are representative, correctness-first reference implementations — not performance-optimized, and not exhaustively handling every edge case in prose form above (e.g., configuration validation, logging calls) for brevity. They are the starting point a developer implements against and writes Section 28 unit tests for, not a drop-in production module.

26 Java Implementation

26.1 Recommended project structure

```
src/  
├── model/  
│   ├── Candle.java  
│   ├── Trade.java  
│   └── Position.java  
├── data/  
│   ├── DataLoader.java  
│   └── DataValidator.java  
├── indicators/  
│   ├── EMA.java  
│   ├── RSI.java  
│   └── ATR.java  
├── strategy/  
│   └── TrendPullbackStrategy.java  
├── execution/  
│   ├── ExecutionSimulator.java  
│   └── CostModel.java  
├── portfolio/  
│   ├── Portfolio.java  
│   └── TradeLedger.java  
├── backtest/  
│   └── Backtester.java  
├── metrics/  
│   └── PerformanceMetrics.java  
├── config/  
│   └── BacktestConfig.java  
└── Main.java
```

26.2 Class relationships

Backtester owns one TrendPullbackStrategy, one Portfolio (which owns the current Position, if any, and the TradeLedger), and drives the Section 8 event loop over a List<Candle>. TrendPullbackStrategy owns instances of EMA, RSI, and ATR (stateful, incrementally-updated indicator objects — see 25.3) and exposes a single method that answers “does bar t produce an entry signal, and if so, what stop/target?” ExecutionSimulator is a stateless utility invoked by Backtester to resolve fills, given CostModel for cost application. This mirrors the Python module boundaries in Section 23 one-for-one, so a reviewer can cross-check the two implementations component by component.

Interfaces are used only where more than one implementation genuinely exists or is anticipated (e.g., an Indicator interface implemented by EMA/RSI/ATR so the IndicatorEngine-equivalent can iterate over them uniformly, and a SlippageModel interface with Fixed/Percentage/BasisPoints implementations per Section 11.1). No interface is introduced merely for the sake of “genuinely enterprise” structure; a single-implementation class stays a plain class.

27 Java Code Requirements — Representative Implementations

27.1 Candle

```
public final class Candle {
    public final long timestampEpochDay;
    public final double open, high, low, close;
    public final long volume;

    public Candle(long timestampEpochDay, double open, double high,
                  double low, double close, long volume) {
        this.timestampEpochDay = timestampEpochDay;
        this.open = open; this.high = high;
        this.low = low; this.close = close;
        this.volume = volume;
    }

    public boolean isValid() {
        if (open <= 0 || high <= 0 || low <= 0 || close <= 0 || volume < 0) return false;
        if (high < low) return false;
        if (high < open || high < close) return false;
        if (low > open || low > close) return false;
        return true;
    }
}
```

27.2 EMA (stateful, incremental — mirrors Section 5.1)

```
public class EMA {
    private final int period;
    private final double alpha;
    private double value = Double.NaN;
    private int seedCount = 0;
```

```

private double seedSum = 0.0;
private boolean ready = false;

public EMA(int period) {
    this.period = period;
    this.alpha = 2.0 / (period + 1);
}

public void update(double close) {
    if (!ready) {
        seedSum += close;
        seedCount++;
        if (seedCount == period) {
            value = seedSum / period;
            ready = true;
        }
        return;
    }
    value = alpha * close + (1 - alpha) * value;
}

public boolean isReady() { return ready; }
public double getValue() { return value; }
}

```

27.3 RSI (Wilder, mirrors Section 5.2)

```

public class RSI {
    private final int period;
    private Double avgGain = null, avgLoss = null;
    private double prevClose = Double.NaN;
    private int count = 0;
    private double gainSum = 0.0, lossSum = 0.0;
    private boolean ready = false;

    public RSI(int period) { this.period = period; }

    public void update(double close) {
        if (Double.isNaN(prevClose)) { prevClose = close; return; }

        double delta = close - prevClose;
        double gain = Math.max(delta, 0.0);
        double loss = Math.max(-delta, 0.0);
        prevClose = close;

        if (!ready) {
            gainSum += gain; lossSum += loss; count++;
            if (count == period) {
                avgGain = gainSum / period;
                avgLoss = lossSum / period;
                ready = true;
            }
        }
    }
}

```

```

        return;
    }
    avgGain = (avgGain * (period - 1) + gain) / period;
    avgLoss = (avgLoss * (period - 1) + loss) / period;
}

public boolean isReady() { return ready; }

public double getValue() {
    if (!ready) return Double.NaN;
    if (avgLoss == 0.0) return 100.0;           // explicit branch – Section 5.2
    double rs = avgGain / avgLoss;
    return 100.0 - (100.0 / (1.0 + rs));
}
}

```

27.4 ATR (Wilder, mirrors Section 5.3)

```

public class ATR {
    private final int period;
    private Double value = null;
    private double prevClose = Double.NaN;
    private int count = 0;
    private double trSum = 0.0;
    private boolean ready = false;

    public ATR(int period) { this.period = period; }

    public void update(double high, double low, double close) {
        double tr;
        if (Double.isNaN(prevClose)) {
            tr = high - low;
        } else {
            tr = Math.max(high - low,
                Math.max(Math.abs(high - prevClose), Math.abs(low - prevClose)));
        }
        prevClose = close;

        if (!ready) {
            trSum += tr; count++;
            if (count == period) { value = trSum / period; ready = true; }
            return;
        }
        value = (value * (period - 1) + tr) / period;
    }

    public boolean isReady() { return ready; }
    public double getValue() { return value; }
}

```

27.5 Strategy — entry signal (mirrors Section 3.3, Python 24.4)

```
public class TrendPullbackStrategy {
    private final EMA ema;
    private final RSI rsi;
    private final ATR atr;
    private final int slopeBars;
    private final double rsiReclaimLevel;
    private final double stopAtrMultiple;
    private final double rewardRiskMultiple;

    private final java.util.Deque<Double> emaHistory = new java.util.ArrayDeque<>();
    private double prevRsi = Double.NaN;

    public TrendPullbackStrategy(int emaPeriod, int slopeBars, int rsiPeriod,
                                double rsiReclaimLevel, int atrPeriod,
                                double stopAtrMultiple, double rewardRiskMultiple) {
        this.ema = new EMA(emaPeriod);
        this.rsi = new RSI(rsiPeriod);
        this.atr = new ATR(atrPeriod);
        this.slopeBars = slopeBars;
        this.rsiReclaimLevel = rsiReclaimLevel;
        this.stopAtrMultiple = stopAtrMultiple;
        this.rewardRiskMultiple = rewardRiskMultiple;
    }

    /** Call once per bar, in chronological order, before checking for a signal. */
    public void onBar(Candle c) {
        ema.update(c.close);
        rsi.update(c.close);
        atr.update(c.high, c.low, c.close);

        if (ema.isReady()) {
            emaHistory.addLast(ema.getValue());
            if (emaHistory.size() > slopeBars + 1) emaHistory.removeFirst();
        }
    }

    /** Must be called after onBar() for the same candle. */
    public Signal evaluate(Candle c) {
        double curRsi = rsi.getValue();
        Signal result = null;

        boolean warmedUp = ema.isReady() && rsi.isReady() && atr.isReady()
            && emaHistory.size() == slopeBars + 1;

        if (warmedUp) {
            double emaNow = emaHistory.peekLast();
            double emaPrior = emaHistory.peekFirst();
            boolean trendOk = (emaNow > emaPrior) && (c.close > emaNow);
            boolean reclaim = !Double.isNaN(prevRsi)
                && prevRsi < rsiReclaimLevel
                && curRsi >= rsiReclaimLevel;
        }
    }
}
```

```

        if (trendOk && reclaim) {
            double riskPerShare = stopAtrMultiple * atr.getValue();
            result = new Signal(riskPerShare, rewardRiskMultiple);
        }
        prevRsi = curRsi;
        return result;
    }

    public static final class Signal {
        public final double riskPerShare;
        public final double rewardRiskMultiple;
        public Signal(double riskPerShare, double rewardRiskMultiple) {
            this.riskPerShare = riskPerShare;
            this.rewardRiskMultiple = rewardRiskMultiple;
        }
    }
}

```

27.6 Correspondence to the Python implementation

Concern	Python (Section 24)	Java (Section 26)
EMA/RSI/ATR calculation method	Vectorized, whole-series (pandas)	Incremental, one bar at a time (streaming-ready)
Warm-up gating	is_ready boolean mask over the series	isReady() per indicator, warmedUp composite flag
RSI zero-avg-loss branch	Explicit if avg_loss == 0: rsi = 100.0	Explicit if (avgLoss == 0.0) return 100.0;
Signal definition	Section 3.3, four AND-ed boolean conditions	Same four conditions, same operators, same boundary semantics (Section 3.2)
Stop/target math	Entry - 1.5×ATR, Entry + 3×ATR	Same formula, computed from riskPerShare in Signal

The vectorized-vs-incremental difference in calculation style is intentional and does not change the specified indicator values (Section 5) — it reflects Python’s strength in offline batch research versus Java’s more natural fit for a bar-by-bar, potentially live-extensible engine (Section 27). Both must produce numerically identical (within floating-point tolerance) EMA/RSI/ATR series on the same input data; this equivalence is exactly what the regression test suite in Section 28 must verify.

28 Python vs. Java — Comparison

Dimension	Python	Java	Notes
Research/iteration speed	High — pandas/NumPy make exploratory analysis, plotting, and rapid rule changes fast	Lower — more boilerplate per change, slower edit-compile-run cycle	Python is the better environment for the exploratory phase where the strategy rules themselves are still being reasoned about
Data manipulation	Excellent (pandas) for vectorized indicator math and tabular trade ledgers	Requires more manual work (no first-class DataFrame equivalent in the standard library)	
Backtesting throughput	Adequate for single-symbol, daily-bar backtests; can become slow for large multi-symbol, high-frequency sweeps unless vectorized carefully	Generally faster raw execution for large-scale, multi-symbol, or intraday-frequency backtests	Matters more at scale (Section 33) than for a single-symbol daily strategy
Visualization	Strong, mature (matplotlib and the broader ecosystem)	Weaker out of the box; typically exports data for an external/Python visualization step	
Maintainability at scale	Dynamic typing can hide errors until runtime in a large codebase	Static typing catches a class of errors at compile time; more verbose but more self-documenting	
OOP / architecture	Supports OOP but often used more script-like in research code	Naturally encourages the class-based structure this specification uses throughout	
Deployment	Simple for internal tools; packaging for external distribution is more friction than Java's	JAR-based deployment is mature and well understood in enterprise/trading-infrastructure contexts	
Fit for future live trading	Workable, but the GIL and dynamic typing are real considerations for latency-sensitive paths	Generally the more natural choice for a low-latency order-management/execution layer (Section 35)	
			Neither language choice here commits the project to a specific broker or execution technology

Dimension	Python	Java	Notes
Developer productivity (solo/small team)	Typically faster to a working first version	Typically slower to a working first version, faster to a maintainable large version	

Recommendation: use Python as the primary research environment (Sections 1–24 of this specification are most naturally prototyped and validated there first), and treat the Java implementation (Sections 25–26) as a secondary, verification-grade reimplement — valuable specifically because getting the same numbers from two independent codebases is strong evidence the logic, not just one language’s library quirks, is correct. **This is a project-development recommendation for this specific codebase and team shape, not a general claim that Python is “better” than Java** — a team already standardized on a Java trading stack would reasonably reverse this recommendation.

29 Testing Strategy

29.1 Indicators

- **EMA correctness:** compare against a known reference series (e.g., hand-computed or a trusted third-party calculation) for at least 30 bars past warm-up; verify the seed value is exactly the SMA of the first period closes, not the first close alone.
- **RSI correctness:** verify the explicit $\text{AvgLoss} == 0 \rightarrow \text{RSI} = 100$ branch with a constructed all-up-bars fixture; verify a constructed all-down-bars fixture yields $\text{RSI} = 0$; verify against a known reference series for the general case.
- **ATR correctness:** verify the True Range formula picks the correct one of the three candidates in constructed fixtures (a gap-up day where $\text{High} - \text{PrevClose}$ is the max component; a gap-down day where $\text{Low} - \text{PrevClose}$ ’s absolute value is the max; a normal day where $\text{High} - \text{Low}$ is the max).

29.2 Strategy

- **Trend condition:** constructed fixture where EMA has risen over 5 bars and price is above it (must fire) vs. EMA flat/declining (must not fire) vs. price above a rising EMA but by using $\text{Close}[t-1]$ instead of $\text{Close}[t]$ (must NOT fire — regression test against the exact Section 3.1 bug this document explicitly prohibits).
- **RSI reclaim boundary:** fixture where $\text{RSI}[t-1] == 40.0$ exactly (must NOT fire, per Section 3.2’s strict <); fixture where $\text{RSI}[t] == 40.0$ exactly after being below (must fire, per the non-strict >=).
- **Invalid signal:** fixture where only one of the two top-level conditions (trend, reclaim) is true (must not fire — regression against accidentally OR-ing instead of AND-ing, Section 3.3).
- **Entry timing:** fixture verifying the order is scheduled at bar t and filled at bar $t+1$ ’s open, never at bar t ’s close (Section 3.4).

29.3 Risk

- **Stop calculation:** $\text{Entry} - 1.5 \times \text{ATR}$ exactly, for a range of ATR values including very small and very large.

- **Target calculation:** $\text{Entry} + 3 \times \text{ATR}$ exactly; assert it always equals $\text{Entry} + 2 \times (\text{Entry} - \text{Stop})$.
- **Position sizing:** fixture producing a non-zero size (Section 12.2); fixture where risk-per-share exceeds the max-risk budget entirely, asserting the zero-size path logs `signal_rejected` rather than silently dropping (Section 12.3); fixture where capital, not risk, is the binding constraint (Section 13).

29.4 Execution

- **Stop hit only:** bar where $\text{Low} \leq \text{Stop}$ and $\text{High} < \text{Target}$ — exit at Stop.
- **Target hit only:** bar where $\text{High} \geq \text{Target}$ and $\text{Low} > \text{Stop}$ — exit at Target.
- **Both hit:** bar satisfying both — assert the conservative policy (Section 9.3) resolves to Stop, and that `ambiguous_exit = true` is recorded.
- **End-of-data:** data ends with a position open — assert exit at last close, `exit_reason = END_OF_DATA`.

29.5 Portfolio

- **P&L:** known entry/exit/quantity fixture, assert `gross_pnl` matches hand calculation.
- **Costs:** known trade value fixture against the Section 10.3 example configuration, assert each cost line item and the GST-on-(brokerage+exchange+SEBI) base is computed correctly.
- **Equity:** assert $\text{Equity} = \text{Starting capital} + \text{Realized P\&L} + \text{Unrealized P\&L}$ holds at every bar, not just at trade close.
- **Drawdown:** constructed equity curve fixture with a known peak-to-trough decline, assert the computed max drawdown matches.

29.6 Regression

Maintain one fixed, versioned, small synthetic OHLCV dataset (not real market data, so it never silently changes) with hand-verified expected indicator values, expected signals, and expected trade ledger. Run this fixture through both the Python and Java implementations in CI; any change to either codebase that alters the result on this fixture must be treated as a deliberate, reviewed logic change — never merged silently.

30 Data Leakage and Look-Ahead Bias

Source	Safeguard
Using future candles to compute a signal (e.g., a centered moving average, or an indicator library that inadvertently uses <code>shift(-1)</code>)	Every indicator function must be reviewed to confirm it only ever indexes bars $\leq t$ when producing the value for bar t ; add a unit test that truncates the series at t and asserts the value for bar t is unchanged
Entering on the same candle as an unexecutable signal (Section 3.4)	Hard-coded <code>next_open</code> entry timing; unit test (Section 28.2) asserting fills never occur at the signal bar's own close

Source	Safeguard
Using future-adjusted corporate action data incorrectly (i.e., a split-adjustment factor computed using information only known after the split, applied to bars before it occurred, in a way that changes historical signal generation retroactively for a “live-as-of-that-date” claim)	Document explicitly (Section 4.7) that fully back-adjusted series are for internal signal-consistency, not a claim of “what a live signal would have shown on that exact date”; do not conflate the two use cases in one report
Using future volatility (e.g., computing ATR over a centered or future window)	Same safeguard as the first row — Wilder ATR as specified is strictly backward-looking by construction
Optimizing parameters on the test/OOS set	Process control (Section 18.2), not a code fix — log every OOS evaluation with a timestamp and require sign-off that no parameter changed since the prior OOS run before treating a result as valid
Using survivorship-biased stock lists	Section 30
Using future index constituents (e.g., testing “NIFTY 50 stocks” using today’s constituent list applied to 10 years ago)	Explicitly flag as a known limitation (Section 30) unless historical constituent data is sourced and used

31 Survivorship Bias

Testing a strategy only on companies that are *still listed and still in a benchmark index today* systematically excludes every company that was delisted, went bankrupt, or was removed from the index for poor performance during the backtest period. Since the strategy is long-only and trend-following, this exclusion tends to inflate the apparent win rate and expectancy, because the very worst historical outcomes (total loss from delisting/bankruptcy) are structurally absent from the test universe by the way that universe was selected — not because the strategy would have avoided them.

Future support requirement (not V1.0): the system should eventually accept a point-in-time constituent list (i.e., “these were the NIFTY 50 members as of this historical date,” including names later delisted or removed) rather than always using today’s list. Until this is implemented, every report produced by this system must state which universe-selection method was used and disclose survivorship bias as a known limitation of any multi-stock result (Section 33).

32 Liquidity Filter (Optional)

Definition, disabled by default:

```
liquidity_filter:
  enabled: false
  min_avg_traded_value_rupees: 10_000_000 # e.g., 20-day average
  min_avg_volume_shares: 100_000
  min_price_rupees: 20.0
```

When enabled, a symbol/bar failing any threshold is excluded from signal generation for that bar (treated as if warm-up were not complete), not retroactively removed from history. This exists to keep the base single-symbol strategy validation (Sections 1–22) free of an additional confound; it becomes relevant once the strategy is extended to a multi-stock universe (Section

33) where illiquid names can otherwise dominate apparent performance through unrealistic assumed fill quality.

33 Market Regime Analysis

Segment the trade ledger and equity curve, after the fact, by regime label, without changing the strategy’s own logic:

Regime	Illustrative classification approach
Bull market	Broad index (e.g., NIFTY 50) above its own 200-day moving average and rising
Bear market	Broad index below its own 200-day moving average and declining
Sideways	Broad index within a defined band around its moving average for an extended period
High volatility	Index or symbol’s own ATR/realized volatility above a rolling historical percentile threshold
Low volatility	Below that threshold

Regime classification is computed on a **separate, independent series** (the benchmark index or a volatility measure) and used only to *label and filter trades after the backtest has already run* — it must never feed back into the Strategy component’s entry/exit logic in V1.0. This keeps the question “does performance concentrate in a particular regime?” (Section 44, Q6) answerable without conflating it with “should the strategy behave differently by regime?”, which is a materially larger, separate design question for a future version.

34 Multi-Stock Backtesting

34.1 Per-symbol independence

For every symbol in the universe:

1. Load and validate that symbol’s OHLCV independently.
2. Compute EMA/RSI/ATR independently, using only that symbol’s own price history.
3. Generate signals independently.
4. Track that symbol’s open position and trade ledger independently (one position per symbol, per Section 3.8 — the one-position rule is per-symbol, not a single global slot, unless the portfolio-level `max_positions` risk control in Section 37 is set to something other than “unlimited across symbols”).
5. Aggregate: combine per-symbol trade ledgers into one portfolio-level ledger and equity curve for Section 15 reporting.

34.2 The critical bug this section exists to prevent

Never concatenate multiple symbols’ OHLCV rows into a single continuous time series and run one indicator calculation over the concatenation. Doing so computes, for example, an “EMA(50)” that blends the tail of one stock’s price history into the head of the next stock’s — a meaningless number, and one that will generate spurious signals at every symbol boundary. Each symbol must have its own, fully independent indicator state (this is

precisely why the Java implementation in Section 26 models indicators as small, per-instance stateful objects — one EMA/RSI/ATR instance per symbol, never shared).

35 Multi-Timeframe Extension (Future Work — Not in V1.0)

V1.0 uses a single timeframe (daily bars) for both trend and entry. Multi-timeframe logic is documented here as a future direction, not implemented.

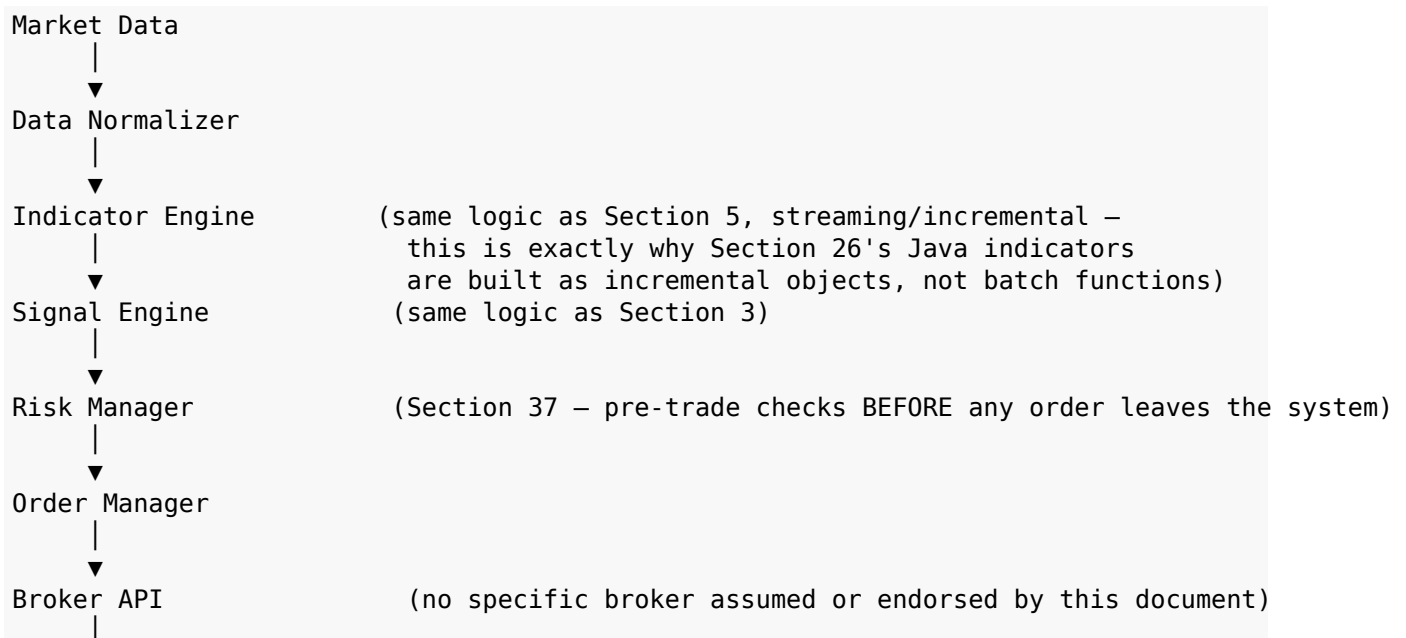
Possible future architecture:

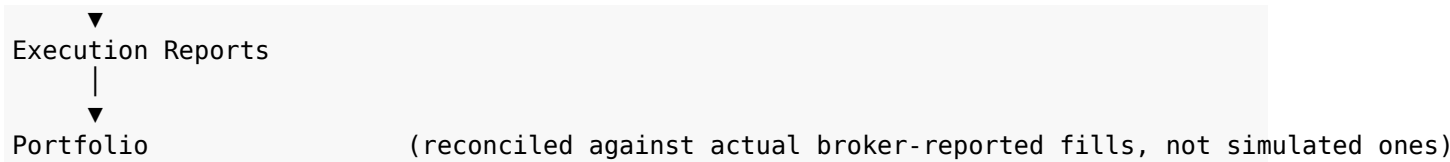
```
Daily trend filter (Section 3.1, unchanged)
+
Hourly pullback confirmation
+
15-minute entry timing refinement
```

Risks of introducing multiple timeframes, stated plainly: (1) it multiplies the look-ahead-bias surface area — aligning a daily trend value with an intraday bar requires care that the daily value used was actually known as of that intraday moment (not the daily bar’s own future close); (2) it multiplies data requirements and cost (intraday history for the full backtest universe); (3) it adds parameters (which faster timeframe, what confirmation rule on it), each of which reopens the overfitting risk in Section 20–21; (4) it makes the strategy harder to reason about and departs from the “no indicator soup” principle stated in Section 1.6 unless the added complexity is justified by a clearly separate, tested hypothesis, not just “more granularity seems like it should help.”

36 Live Trading Architecture (Future Work — Not in V1.0)

No live trading, order routing, or broker connectivity exists in V1.0. The following is documented as a target architecture for a future phase (Section 42, Phase 13), not a current capability.





No broker is named or assumed. Any broker mentioned elsewhere as an example is illustrative only and implies no endorsement, integration guarantee, or compatibility claim.

37 Paper Trading (Future Work — Not in V1.0)

A required phase (Section 42, Phase 12) **before any real capital is committed**, once live infrastructure (Section 35) exists:

- **Signal logging** — every signal the live system generates, timestamped, whether or not it is acted on.
- **Simulated execution** — orders are generated and priced as if filled, using real-time market data, but never actually routed to a broker.
- **Slippage tracking** — compare the simulated fill price against the actual best available price at that moment, to calibrate the Section 11 slippage model against real (not back-tested) market microstructure.
- **Latency tracking** — measure the time from signal generation to (simulated) order placement; this is a real cost the historical daily-bar backtest cannot measure at all, since it has no concept of intraday latency.
- **Difference between expected and actual fills** — the single most important paper-trading output: if simulated fills systematically differ from the backtest’s slippage assumptions, the Section 11 model must be recalibrated before real capital is at risk.
- **Strategy drift detection** — ongoing comparison of live/paper signal frequency and character against the historical backtest’s signal frequency and character, to catch data-source or environment discrepancies early.

38 Risk Management

System-level controls, all configurable (Section 38), enforced by the RiskManager component (Section 7.1):

Control	Purpose
Maximum risk per trade	Caps rupee loss on any single position (Section 12.2’s basis)
Maximum daily loss	Halts new entries for the remainder of the day/session if realized+unrealized losses exceed a threshold
Maximum open positions	Caps concurrent exposure, especially relevant once multi-stock (Section 33) is enabled
Maximum portfolio exposure	Caps total capital committed to open positions as a fraction of equity
Maximum drawdown threshold	A circuit breaker — halts new entries (does not force-close existing positions) if running drawdown from peak equity exceeds a configured limit, pending manual review

Control	Purpose
Trading halt conditions	E.g., exchange-wide circuit filter events, stale/missing data feed
Data failure handling	Defined fallback: skip signal evaluation for affected symbol/bar rather than proceeding on stale or partial data
API failure handling	Applicable only once live (Section 35): defined retry/backoff and fail-safe (do not silently retry an order submission in a way that could double-submit)

These are described here as requirements for the eventual live/paper system; in the V1.0 backtester, only the position-sizing and max-positions controls are directly exercised (Sections 12-13), and the others are validated structurally (configuration exists, is read, is enforced in code) even though a historical backtest has no “daily loss halts new entries for the rest of the day” real-time concept in the same way a live system does — the backtest’s equivalent is simply not opening a new position on a day where the daily-loss limit was already breached earlier that same day, which requires intraday sequencing the daily-bar backtest does not have. This is flagged explicitly as a limitation: **the base V1.0 daily-bar backtester validates these risk controls at the level of “per bar,” not “intraday,” and this gap is closed only once intraday or live data is used.**

39 Configuration

```

strategy:
  ema_period: 50
  slope_bars: 5
  rsi_period: 14
  rsi_level: 40
  atr_period: 14
  stop_atr: 1.5
  reward_risk: 2.0

execution:
  entry: next_open           # only supported value in V1.0
  intrabar_policy: conservative # Section 9.3; only supported value in V1.0
  slippage_type: percentage   # fixed | percentage | bps
  slippage_value: 0.05

cost_model:
  brokerage_type: percentage
  brokerage_value: 0.0
  exchange_transaction_charge_pct: 0.00297 # illustrative – verify before use, Section 10.2
  stt_pct_delivery: 0.10                 # illustrative
  sebi_charges_per_crore: 10             # illustrative
  stamp_duty_pct_buy_leg: 0.015          # illustrative
  gst_pct: 18.0                          # illustrative

risk:
  max_risk_per_trade: 1000
  max_positions: 1

```

```

max_daily_loss: 5000
max_drawdown_pct: 20.0

capital:
  starting_capital: 500000

data:
  frequency: daily
  min_history_bars: 250
  adjustment: split_and_bonus_adjusted # Section 4.7

```

Parameter	Meaning
strategy.*	Section 3 / Section 20 base parameters and sweep anchors
execution.entry	Fill timing rule (Section 3.4)
execution.intrabar_policy	Section 9.3 ambiguity resolution
execution.slippage_*	Section 11
cost_model.*	Section 10 — all illustrative, must be verified before real use
risk.*	Section 37
capital.starting_capital	Section 13
data.*	Sections 4, 6

40 Logging

Structured (machine-parseable, e.g., JSON-lines) log events required for:

Event category	Minimum fields
Data errors	symbol, timestamp, rule violated (Section 4.6), raw values
Signals	symbol, signal_timestamp, indicator values at signal, resulting stop/target
Orders	symbol, order type (entry/exit), scheduled bar, resolved fill bar
Entries	trade_id, entry_timestamp, entry_price, quantity
Exits	trade_id, exit_timestamp, exit_price, exit_reason, ambiguous_exit flag
Risk violations	which control (Section 37) fired, symbol, timestamp, values that triggered it
Backtest events	run start/end, configuration hash (Section 47), data range
Exceptions	full stack trace, last successfully processed bar, component

Logs are the primary audit trail supporting Section 41’s auditability acceptance criterion — every trade in the TradeLedger must be traceable back to a specific signal log entry and the exact configuration in force at that time.

41 Output Report

41.1 Required structure

Executive Summary — period, symbol(s), timeframe, starting capital, ending capital, net return, maximum drawdown, number of trades, win rate, profit factor, expectancy, plus one line explicitly stating the Section 9.3 intrabar-ambiguity policy and Section 10.2 illustrative-cost-rate caveat used for this run.

Strategy Configuration — every parameter from Section 38, verbatim, plus a configuration hash (Section 47) so the exact run is reproducible.

Trade Statistics — full Section 15 metrics table.

Risk Statistics — drawdown curve summary, drawdown duration, Monte Carlo percentile outputs (Section 22.2) if that module was run for this report.

Charts — equity curve, drawdown curve, trade P&L distribution, monthly returns heatmap (Section 16).

Validation — in-sample vs. out-of-sample comparison (Section 18), walk-forward summary (Section 19) if run, sensitivity plateau/spike assessment (Section 20) if run, Monte Carlo summary (Section 22) if run. **If any of these were not run for a given report, the report must say so explicitly** (“Walk-forward analysis: not performed for this run”) rather than omit the section silently, so a reader cannot mistake an unvalidated single backtest for a validated one.

42 Acceptance Criteria

42.1 Correctness

- No look-ahead bias: entry fills never use the signal bar’s own close (Section 3.4); verified by the Section 28.2 entry-timing test.
- No data leakage: indicator calculations use only current-and-past bars (Section 29); verified by truncation tests.
- Indicator calculations match a hand-verified reference series within floating-point tolerance (Section 28.1).
- Entry occurs only after signal confirmation, on the following bar’s open, never earlier.
- Stop and target are calculated exactly per Sections 3.5–3.6, verified by unit test.

42.2 Performance reporting

All Section 15 metrics, all Section 16 visualizations, and the full Section 40 report structure are generated for every backtest run, with no metric silently omitted because it is inconvenient (e.g., an undefined/infinite profit factor must be reported as such, not hidden).

42.3 Reproducibility

Identical dataset + identical configuration (including the illustrative cost rates actually used, not just referenced) must produce an identical trade ledger and identical metrics, bit-for-bit or within a documented floating-point tolerance, across repeated runs and across the Python and Java implementations.

42.4 Testing

Every component listed in Section 7.1 has automated tests per Section 28; CI must run the full test suite, including the fixed regression dataset, on every change.

42.5 Auditability

Every trade in the final ledger must be reconstructable from the raw input data plus the run's configuration alone — no trade should depend on any manual step, hidden default, or undocumented override.

42.6 What is explicitly NOT an acceptance criterion

Profitability is not an acceptance criterion. A correctly implemented, fully tested, fully validated system that concludes the strategy has no edge (Section 45) has still met every acceptance criterion in this section and is a complete, successful deliverable. Do not gate project completion on the backtest “looking good” — gate it on the backtest being *correct and honestly reported*, whatever it shows.

43 Project Milestones

Phase	Deliverable	Dependencies	Definition of done
1. Data ingestion	DataLoader, DataValidator	—	Loads and validates a real historical OHLCV file per Section 4 rules; rejection logging works
2. Indicators	EMA/RSI/ATR (Section 5)	Phase 1	Matches hand-verified reference series; warm-up (Section 6) enforced
3. Signal engine	Strategy (Section 3)	Phase 2	Section 28.2 tests pass, including the boundary and anti-lookahead regression tests
4. Backtester	Full Section 8 event loop	Phase 3	Runs end-to-end on a real symbol, produces a raw (no-cost) trade ledger
5. Costs and slippage	CostModel, slippage (Sections 10–11)	Phase 4	Net-of-cost ledger matches hand-verified fixture (Section 28.5)
6. Performance reporting	PerformanceAnalyzer, charts (Sections 15–16)	Phase 5	Full metrics table and chart set generated from a real backtest

Phase	Deliverable	Dependencies	Definition of done
7. Testing	Full Section 28 suite, CI wired up	Phases 1–6	All tests passing; regression fixture locked
8. Out-of-sample validation	Section 18 protocol executed once	Phase 7	OOS result logged exactly once with timestamp (Section 47)
9. Walk-forward analysis	Section 19 framework	Phase 7	Per-window and aggregated results reported
10. Python completion	All of Sections 23–24 implemented and tested	Phases 1–9	Reference implementation frozen and tagged
11. Java reimplementation	All of Sections 25–26 implemented and tested	Phase 10	Section 28.6 regression fixture produces matching results in both languages
12. Paper trading	Section 36	Phase 11 + live data feed (out of this document’s scope to build)	Signal/slippage/latency logs collected over a defined trial period
13. Optional live integration	Section 35	Phase 12, explicit go/no-go decision based on paper-trading results	Broker connectivity, order management — only if Phase 12 supports proceeding

44 Versioning

Version	Scope
V1.0	EMA trend filter, RSI reclaim trigger, ATR stop, 2R target, long-only, single position, historical backtesting, no costs modeled yet (or costs at a minimal/zero default)
V1.1	Adds full transaction cost model (Section 10), slippage model (Section 11), improved reporting (Section 40)
V1.2	Adds walk-forward testing (Section 19), Monte Carlo analysis (Section 22), parameter sensitivity sweeps (Section 20)
V2.0 (candidate scope)	Portfolio-level multi-symbol execution (Section 33), regime analysis (Section 32), paper trading (Section 36)

Features are added only when they answer a specific open question from Section 44 or close a specific gap identified in validation — not to increase project scope for its own sake. A trailing stop, for instance, is a legitimate future strategy variant, but it is a **different strategy** than the one specified in Section 3 and must be versioned and validated as such (e.g., “V1.3 — trailing stop variant”), never silently merged into the V1.0 baseline results.

45 Research Questions

The project is not “done” when the code runs — it is done when these are answered, with evidence, in the final report:

1. Does the strategy have positive expectancy, net of costs and slippage?
2. Is the result robust after realistic transaction costs (Section 10)?
3. Is the result robust after realistic slippage (Section 11)?
4. Does performance survive out-of-sample testing (Section 18)?
5. Is performance concentrated in a small number of outlier trades, or broad-based (Section 16, R-multiple distribution)?
6. Is performance concentrated in a particular market regime (Section 32)?
7. Is the strategy sensitive to the RSI reclaim threshold (Section 20)?
8. Is the strategy sensitive to the ATR stop multiple (Section 20)?
9. Does the edge survive across multiple stocks, not just the one it was designed against (Section 33)?
10. Does the edge survive across different historical time periods?
11. What is the maximum historical drawdown, and what does Monte Carlo suggest about plausible future drawdowns (Section 22)?
12. What happens to the result under deliberately adverse execution assumptions (worse slippage, less favorable intrabar resolution — Section 22.1)?
13. Does parameter optimization (Section 20) materially improve results relative to the a priori base parameters?
14. If it does, is that improvement stable across the sensitivity sweep (a plateau) or a narrow spike indicating overfitting (Section 20.4)?

46 Failure Conditions

The research must conclude the strategy, as specified, requires revision or rejection if **any** of the following are found:

Failure condition	Required action
Negative expectancy net of costs	Do not proceed to live/paper trading; either revise the hypothesis (Section 2) or stop
Profit factor at or below 1.0 out-of-sample	Same as above
Excessive drawdown relative to the strategy’s own risk budget	Investigate whether position sizing (Section 12) or the stop distance (Section 3.5) is the cause before concluding the entry logic itself is at fault
Severe parameter sensitivity (a spike, not a plateau — Section 20.4)	Treat the base-parameter result as likely overfit; do not simply pick a different single parameter combination and re-declare success
Large performance deterioration out-of-sample vs. in-sample/validation	The in-sample result was likely fit to that period’s noise; do not average the two together to produce a more favorable blended number
Performance disappears after realistic costs/slippage (Section 21, cost/slippage sensitivity)	The strategy may only “work” in a frictionless model that does not correspond to reality
Performance concentrated in a handful of trades (Section 44, Q5)	Report this explicitly; a strategy that depends on a few outlier trades is fragile even if its headline numbers look fine

Failure condition	Required action
Monte Carlo shows an unacceptable probability of ruin or drawdown at the intended position sizing (Section 22.2)	Reduce position size, revise risk controls (Section 37), or do not proceed
Strategy only “works” under unrealistic execution assumptions (e.g., filling at the signal bar’s own close, or ignoring the Section 9 intrabar ambiguity)	This indicates the backtest itself may be biased, not that the strategy has a real edge

The correct response to any of these is to report the finding, not to adjust parameters or assumptions until the result looks better. Doing the latter is exactly the overfitting process Sections 18 and 20 exist to prevent, applied after the fact instead of before.

47 Security and Reliability

Concern	Handling
Corrupted data	Reject at DataValidator (Section 4.6); never attempt to “repair” a corrupted bar by guessing a plausible value
Missing data	Section 4.4 — reject or (only for confirmed source dropouts) forward-fill with explicit logging
API failures (future, live/paper only)	Defined retry/backoff; never silently proceed as if the last-known data were current
Duplicate records	Section 4.5 — reject the batch, do not silently deduplicate
Timestamp errors	Validate monotonic, strictly increasing timestamps per symbol on load; reject out-of-order or duplicate timestamps
Timezone errors	Standardize on a single timezone (exchange local time, IST for NSE/BSE) at ingestion; never mix naive and timezone-aware timestamps in the same series
Floating-point precision	Use double/64-bit floats consistently for prices; be aware that Wilder smoothing recursions can accumulate small floating-point drift over very long series — the regression fixture (Section 28.6) exists partly to catch unexpected drift between implementations
Incorrect corporate actions	Cross-check adjusted-series discontinuities against a known corporate-actions calendar where available; flag unexplained large single-bar moves for manual review rather than trading through them blindly
Partial datasets	Refuse to run a backtest below the Section 4.3 minimum history requirement; fail loudly, not silently with degraded results

Concern	Handling
Configuration errors	Validate the full configuration schema (Section 38) at startup — invalid/missing/out-of-range values must halt the run before any data is processed, not fail midway through

48 Reproducibility

Every backtest run must record, alongside the report (Section 40):

- Dataset version/identifier and source
- Exact date range used
- Timeframe
- Strategy version (Section 43)
- Full configuration (Section 38), as actually used — not a reference to a config file that might have since changed
- Code version / Git commit hash, for both the code and, if separately versioned, the configuration
- Backtest run timestamp
- Cost assumptions actually applied (Section 10.3) — explicitly, not by reference, since illustrative defaults may have been edited
- Slippage assumptions actually applied (Section 11)

A backtest without reproducibility is basically a screenshot of a spreadsheet wearing a lab coat.

This is not decorative. Two backtests reported six months apart, run against “the same strategy,” are not comparable at all unless every item above is known to match — and in practice, if illustrative cost rates were quietly updated, they usually won’t match, which is exactly the kind of silent discrepancy this record exists to catch.

49 Final Project Directory (Both Languages)

```

project-root/
├── python/
│   ├── quant_strategy/                (Section 23.1 structure, in full)
│   │   ├── data/ indicators/ strategy/ backtest/ execution/
│   │   ├── portfolio/ metrics/ visualization/ reports/ tests/
│   │   └── config/ main.py
│   ├── java/
│   │   ├── src/                      (Section 25.1 structure, in full)
│   │   │   ├── model/ data/ indicators/ strategy/ execution/
│   │   │   └── portfolio/ backtest/ metrics/ config/ Main.java
│   └── data/
│       ├── raw/                      (source OHLCV, untouched)
│       ├── regression_fixture/        (Section 28.6 fixed synthetic dataset)
│       └── README.md                  (data provenance, adjustment convention used)
└── reports/

```

```

├── (generated backtest reports, timestamped, never overwritten)
├── docs/
│   ├── this specification, plus any deviation log (Section 3.2, 9.3, etc.)
└── README.md

```

50 Example End-to-End Run (Hypothetical Numbers Only)

Given:

```

Close[t]           = 500.00
EMA50[t]           = 490.00
EMA50[t-5]         = 485.00
RSI14[t-1]         = 37.00
RSI14[t]           = 42.00
ATR14[t]           = 5.00

```

Step 1 — Trend condition: $490.00 > 485.00$ (true) and $500.00 > 490.00$ (true), so trend OK.

Step 2 — RSI reclaim: $37.00 < 40$ (true) and $42.00 \geq 40$ (true), so reclaim confirmed.

Step 3 — Signal: both conditions true $\rightarrow$ long signal fires at bar t .

Step 4 — Entry (next bar): assume $\text{Open}[t+1] = 501.00$. With 0.05% percentage slippage (Section 11): entry fill = $501.00 \times 1.0005 = 501.2505$, i.e. approximately ₹501.25.

Step 5 — Stop: $\text{Stop} = 500.00 - 1.5 \times 5.00 = 492.50$, i.e. ₹492.50 (computed from the signal-bar entry reference price and ATR, per Section 3.5 — using ₹500.00, the signal-bar close, as the entry reference for stop/target math, fixed before the actual fill is known).

Step 6 — Target: $\text{Target} = 500.00 + 3 \times 5.00 = 515.00$, i.e. ₹515.00.

Step 7 — Position sizing: assume max risk = ₹1,000, risk per share = $500.00 - 492.50 = 7.50$ (₹7.50). Position size = $\lfloor 1000/7.50 \rfloor = 133$ shares (Section 12.2).

Step 8 — Costs (illustrative, Section 10.3 rates): trade value (buy leg) = $133 \times 501.25 = 66,666.25$, i.e. ₹66,666.25. Approximate combined statutory + GST cost on this leg, at the illustrative rates in Section 10.3, is on the order of a few tens of rupees per leg — the exact figure must come from running `compute_trade_costs` (Section 24.7) against the actual configured rates, not from a hand-estimated number restated here as fact.

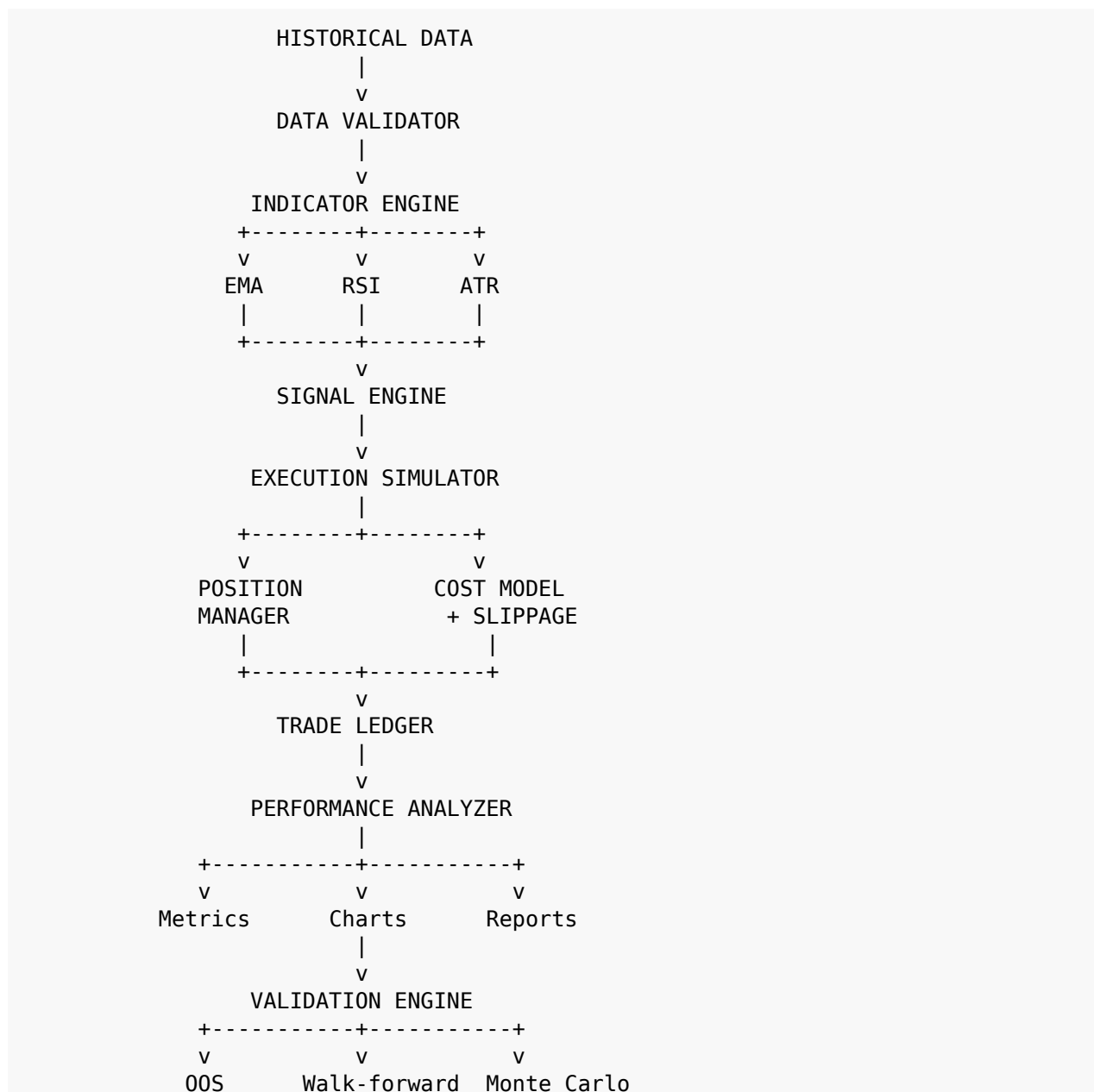
Step 9 — Exit: assume, several bars later, High touches ₹515.00 with Low staying above ₹492.50 on the same bar (unambiguous target hit). Exit fill, with slippage applied against the trader per Section 11.2: $515.00 \times (1 - 0.0005) \approx 514.74$, i.e. approximately ₹514.74.

Step 10 — Gross P&L: $(514.74 - 501.25) \times 133 \approx 1,794.17$, i.e. approximately ₹1,794.17.

Step 11 — R-multiple: risk taken was $7.50 \times 133 = 997.50$ (₹997.50); realized net P&L (gross minus the Step 8 costs on both legs) divided by that risk gives the trade's R-multiple — mechanically greater than 2R gross (since the fill was better than the theoretical entry/target reference in this particular hypothetical direction of slippage on entry, and costs reduce it somewhat), illustrating why the **realized** reward:risk in the TradeLedger (Section 14) is expected to differ from the **theoretical** 2R design target, and why Section 15 reports both.

All figures in this section are illustrative arithmetic on invented numbers, not a claim about any real backtest result.

51 Final Architecture Diagram



52 Glossary

Term	Definition
EMA	Exponential Moving Average — a weighted average of price giving more weight to recent bars, per Section 5.1

Term	Definition
RSI	Relative Strength Index — a momentum oscillator between 0–100 based on the ratio of average gains to average losses (Section 5.2)
ATR	Average True Range — a Wilder-smoothed measure of recent volatility (Section 5.3)
OHLCV	Open, High, Low, Close, Volume — the standard fields of one price bar
R multiple	A trade’s net P&L expressed as a multiple of the initial rupee risk taken (risk per share × quantity)
Expectancy	The average net P&L per trade, combining win rate and average win/loss size (Section 15)
Profit factor	Gross profit divided by the absolute value of gross loss
Drawdown	The decline in equity from a prior peak to a subsequent trough
Sharpe ratio	Excess return per unit of total volatility (standard deviation) of returns
Sortino ratio	Excess return per unit of downside volatility only
Slippage	The difference between a theoretical signal/order price and the price actually achievable in real execution
Look-ahead bias	Using information in a backtest that would not actually have been available at the simulated decision time
Survivorship bias	Distortion caused by testing only on entities (stocks, index members) that “survived” to the present
Data leakage	Any pathway, direct or indirect, by which future information influences a supposedly historical decision
Overfitting	Tuning a strategy’s rules/parameters to fit the noise of a specific historical sample rather than a real, repeatable effect
Walk-forward analysis	Repeated train-then-test cycles across sequential, non-overlapping (or rolling) time windows (Section 19)
Out-of-sample testing	Evaluating a finalized strategy on data it was never used to design or tune against
Monte Carlo simulation	Randomized resampling/perturbation of historical outcomes to characterize a range of plausible results, not a single point estimate

A Full Illustrative Configuration Reference

See Section 38 for the complete annotated YAML configuration schema. Reproduced in full here for convenience of a reader implementing directly from the appendix:

```

strategy:
  ema_period: 50
  slope_bars: 5
  rsi_period: 14
  rsi_level: 40
  atr_period: 14
  stop_atr: 1.5
  reward_risk: 2.0

execution:
  entry: next_open
  intrabar_policy: conservative
  slippage_type: percentage
  slippage_value: 0.05

cost_model:
  brokerage_type: percentage
  brokerage_value: 0.0
  exchange_transaction_charge_pct: 0.00297
  stt_pct_delivery: 0.10
  sebi_charges_per_crore: 10
  stamp_duty_pct_buy_leg: 0.015
  gst_pct: 18.0

risk:
  max_risk_per_trade: 1000
  max_positions: 1
  max_daily_loss: 5000
  max_drawdown_pct: 20.0

capital:
  starting_capital: 500000

data:
  frequency: daily
  min_history_bars: 250
  adjustment: split_and_bonus_adjusted

```

Note repeated deliberately one final time: every rate in the `cost_model` block above is illustrative, reflecting the general shape of NSE/SEBI/statutory charges as commonly published at research time, not a value to trade on without independent verification.

B Items Marked TBD in This Specification

- Specific data vendor/provider (Section 4, intentionally unspecified).
- Specific broker API for eventual live integration (Section 35, intentionally unspecified).
- Document author and reviewer names (Document Control section).
- Exact historical constituent data source for future survivorship-bias-corrected multi-stock testing (Section 30).
- Final numeric brokerage/statutory rates for any real cost analysis (Section 10.2 — must be sourced fresh at time of use, not taken from this document).

References

This document is an original technical specification synthesized from standard, well-established definitions in technical analysis (Wilder's RSI/ATR), quantitative backtesting practice (walk-forward analysis, Monte Carlo resampling, out-of-sample validation), and publicly available NSE/SEBI fee-schedule structures current as of research time (Section 10.3). No academic papers are cited because none were consulted in producing the strategy rules themselves, which are specified directly rather than derived from a specific published study; no citation is fabricated to suggest otherwise.